

SECURE-WEB-LOCK : CLIENT-SIDE FILE ENCRYPTION
IMPLEMENTASI WEB CRYPTO API DENGAN ALGORITMA AES-GCM 256-
BIT DAN PBKDF2



Oleh:

M. Rizki Juniardi	(F1D02310074)
Muhammad Alafath Mavianza	(F1D02310077)
Muzayyan Rozaan	(F1D02310010)

UNIVERSITAS MATARAM
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INFORMATIKA
2026

Daftar Isi

BAB I PENDAHULUAN.....	1
1.1 Latar belakang.....	1
1.2 Rumusan Masalah.....	2
1.3 Batasan Masalah	2
1.4 Tujuan Penelitian	3
1.5 Manfaat Penelitian	3
1.5.1 Manfaat Teoritis.....	3
1.5.2 Manfaat Praktis	3
1.6 Metodologi Pengembangan Sistem.....	3
BAB II LANDASAN TEORI.....	5
2.1 Kriptografi.....	5
2.2 Kriptografi Simetris	5
2.3 <i>Advanced Encryption Standard (AES)</i>	6
2.4 Galois/Counter Mode (GCM).....	6
2.5 <i>Password-Based Key Derivation Function 2 (PBKDF 2)</i>	7
2.6 <i>Secure Hash Algorithm 256 (SHA-256)</i>	8
2.7 <i>Client-Side Encryption</i>	8
2.8 <i>Zero-Knowledge Architecture</i>	8
2.9 Web Crypto API	9
2.10 React	9
2.11 Vite.....	10
2.12 Penelitian Terdahulu	10
BAB III ANALISIS DAN PERANCANGAN SISTEM.....	11
3.1 Analisis Kebutuhan Sistem	11
3.1.1 Kebutuhan Fungsional	11
3.1.2 Kebutuhan Non-Fungsional	11

3.2	Analisis Aktor	11
3.3	<i>Use Case</i> Sistem	12
3.4	Perancangan Arsitektur Sistem	14
3.5	Perancangan Proses Enkripsi	15
3.6	Perancangan Proses Dekripsi	17
BAB IV IMPLEMENTASI DAN PENGUJIAN.....		20
4.1	Implementasi Sistem.....	20
4.2	Pengujian Fungsional.....	23
4.3	Analisis Hasil pengujian	25
BAB 5 PENUTUP		27
5.1	Kesimpulan	27
5.2	Saran	27
DAFTAR PUSTAKA		
LAMPIRAN.....		

BAB I PENDAHULUAN

1.1 Latar belakang

Perkembangan teknologi informasi telah mendorong peningkatan aktivitas pertukaran data digital dalam berbagai sektor, mulai dari pendidikan, pemerintahan, bisnis, hingga aktivitas pribadi. Berbagai layanan penyimpanan awan dan platform berbagi berkas memungkinkan pengguna untuk mengirimkan dokumen dengan cepat dan mudah melalui jaringan internet. Namun, kemudahan tersebut juga meningkatkan risiko keamanan informasi, seperti pencurian data, penyadapan komunikasi, manipulasi berkas, maupun kebocoran data akibat serangan siber [13].

Menurut berbagai laporan keamanan siber dalam beberapa tahun terakhir, kebocoran data masih menjadi salah satu ancaman utama yang dihadapi organisasi maupun individu [14]. Banyak layanan penyimpanan data menerapkan mekanisme enkripsi di sisi server (*server-side encryption*), yaitu data baru dienkripsi setelah diterima oleh server penyedia layanan. Pendekatan ini memang mampu meningkatkan keamanan penyimpanan data, namun penyedia layanan masih memiliki kemungkinan untuk mengakses data pengguna karena kunci enkripsi umumnya berada di bawah kendali server.

Sebagai alternatif yang lebih aman, dikembangkan pendekatan *client-side encryption*, yaitu proses enkripsi dilakukan langsung pada perangkat pengguna sebelum data dikirim atau disimpan. Dengan pendekatan ini, data dalam bentuk asli (*plaintext*) tidak pernah meninggalkan perangkat pengguna sehingga tingkat privasi dan keamanan menjadi lebih tinggi. Konsep tersebut dikenal pula sebagai *Zero-Knowledge Architecture*, yaitu kondisi ketika penyedia layanan tidak memiliki kemampuan untuk mengetahui isi data maupun kunci yang digunakan pengguna.

Perkembangan teknologi web modern memungkinkan implementasi mekanisme kriptografi secara langsung pada browser melalui Web Crypto API. Web Crypto API menyediakan berbagai fungsi kriptografi tingkat rendah seperti pembangkitan bilangan acak, derivasi kunci, enkripsi, dan dekripsi yang dieksekusi menggunakan mekanisme keamanan bawaan browser. Teknologi ini memungkinkan pengembangan aplikasi keamanan data berbasis web tanpa memerlukan instalasi perangkat lunak tambahan. [13] [14].

Dalam proyek ini dikembangkan aplikasi Secure-Web-Lock, yaitu aplikasi enkripsi berkas berbasis web yang menerapkan algoritma *Advanced Encryption Standard* (AES) dengan mode operasi *Galois/Counter Mode* (GCM) dan fungsi derivasi kunci *Password-Based Key Derivation Function 2* (PBKDF2). AES-GCM dipilih karena mampu menyediakan kerahasiaan

data sekaligus menjamin integritas data melalui *Authentication Tag*. Sementara itu, PBKDF2 digunakan untuk menghasilkan kunci kriptografi yang kuat dari kata sandi pengguna sehingga lebih tahan terhadap serangan brute force dan dictionary attack.

Melalui pengembangan Secure-Web-Lock diharapkan dapat dihasilkan aplikasi keamanan data yang mudah digunakan, tidak bergantung pada server untuk proses kriptografi, serta mampu memberikan perlindungan terhadap kerahasiaan dan integritas berkas pengguna.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah:

- a. Bagaimana merancang dan membangun aplikasi enkripsi berkas berbasis web yang menerapkan konsep *client-side encryption*?
- b. Bagaimana mengimplementasikan algoritma AES-GCM 256-bit pada aplikasi web untuk menjamin kerahasiaan dan integritas data?
- c. Bagaimana mengimplementasikan PBKDF2 sebagai mekanisme pembentukan kunci kriptografi yang aman dari kata sandi pengguna?
- d. Bagaimana menguji keberhasilan proses enkripsi dan dekripsi pada berbagai jenis berkas menggunakan Web Crypto API?

1.3 Batasan Masalah

Agar penelitian lebih terarah dan sesuai dengan tujuan yang ingin dicapai, maka ditetapkan batasan masalah sebagai berikut:

- a. Sistem dikembangkan dalam bentuk aplikasi web menggunakan React dan Vite.
- b. Proses enkripsi dan dekripsi dilakukan sepenuhnya pada sisi klien menggunakan Web Crypto API.
- c. Algoritma enkripsi yang digunakan adalah AES-GCM dengan panjang kunci 256-bit.
- d. Pembentukan kunci dilakukan menggunakan PBKDF2 dengan fungsi hash SHA-256.
- e. Sistem tidak menyimpan *password* pengguna pada server maupun basis data.
- f. Sistem tidak menyediakan mekanisme pemulihan *password* apabila pengguna lupa password yang digunakan.
- g. Pengujian sistem difokuskan pada keberhasilan pemrosesan enkripsi dan dekripsi berkas, serta evaluasi analisis keamanan informasi yang meliputi pembuktian arsitektur *Zero-Knowledge* (pengujian lalu lintas jaringan), ketahanan *anti-tampering* (manipulasi biner berkas), dan ketiadaan residu memori sensitif (*memory wiping*) pada peramban lokal.

1.4 Tujuan Penelitian

Tujuan yang ingin dicapai dalam pengembangan aplikasi Secure-Web-Lock adalah sebagai berikut:

- a. Merancang dan membangun aplikasi enkripsi berkas berbasis web menggunakan teknologi Web Crypto API.
- b. Mengimplementasikan algoritma AES-GCM 256-bit untuk menjaga kerahasiaan dan integritas data pengguna.
- c. Mengimplementasikan PBKDF2 sebagai mekanisme pembentukan kunci kriptografi yang aman dari kata sandi pengguna.
- d. Menerapkan konsep *Zero-Knowledge Architecture* melalui pendekatan *client-side encryption*.
- e. Mengevaluasi keberhasilan operasional sistem dalam memproses berbagai jenis berkas digital, sekaligus membuktikan ketahanan keamanan sistem melalui pengujian lalu lintas jaringan, uji manipulasi berkas, dan inspeksi manajemen memori lokal peramban.

1.5 Manfaat Penelitian

1.5.1 Manfaat Teoritis

- a. Menambah wawasan mengenai penerapan kriptografi modern pada aplikasi berbasis web.
- b. Menjadi referensi bagi penelitian selanjutnya yang berkaitan dengan keamanan data dan client-side encryption.
- c. Memberikan pemahaman mengenai implementasi AES-GCM dan PBKDF2 menggunakan Web Crypto API.

1.5.2 Manfaat Praktis

- a. Membantu pengguna melindungi kerahasiaan berkas sebelum dibagikan melalui internet.
- b. Mengurangi risiko kebocoran data akibat akses tidak sah dari pihak ketiga.
- c. Menyediakan solusi keamanan data yang dapat digunakan tanpa instalasi perangkat lunak tambahan.
- d. Meningkatkan kesadaran pengguna mengenai pentingnya keamanan informasi digital.

1.6 Metodologi Pengembangan Sistem

Metode pengembangan sistem yang digunakan dalam proyek ini adalah metode Prototype. Metode ini dipilih karena memungkinkan pengembang memperoleh umpan balik secara cepat dari pengguna terkait kebutuhan antarmuka maupun fungsi sistem.

Tahapan pengembangan meliputi:

- a. Analisis kebutuhan.
- b. Perancangan prototype.
- c. Implementasi sistem.
- d. Pengujian sistem.
- a. Evaluasi dan penyempurnaan.

BAB II LANDASAN TEORI

2.1 Kriptografi

Kriptografi merupakan ilmu yang mempelajari teknik untuk melindungi informasi sehingga hanya pihak yang berwenang yang dapat mengakses informasi tersebut. Kriptografi bertujuan untuk menjaga kerahasiaan, integritas, autentikasi, dan *non-repudiation* suatu data [1].

Dalam sistem keamanan informasi modern, kriptografi digunakan untuk melindungi data selama proses penyimpanan maupun transmisi. Penerapan kriptografi banyak ditemukan pada aplikasi perbankan, layanan penyimpanan awan, komunikasi digital, hingga sistem keamanan data pribadi.

Secara umum, proses kriptografi terdiri atas dua mekanisme utama, yaitu enkripsi dan dekripsi. Enkripsi merupakan proses mengubah data asli (*plaintext*) menjadi bentuk yang tidak dapat dibaca (*ciphertext*), sedangkan dekripsi merupakan proses mengembalikan *ciphertext* menjadi *plaintext* menggunakan kunci tertentu [8].

Tujuan utama kriptografi meliputi:

- a. Kerahasiaan (*Confidentiality*)
- b. Integritas (*Integrity*)
- c. Autentikasi (*Authentication*)
- d. *Non-Repudiation*

Keempat aspek tersebut menjadi fondasi dalam Pembangunan system keamanan informasi modern.

2.2 Kriptografi Simetris

Kriptografi simetris merupakan metode kriptografi yang menggunakan kunci yang sama untuk proses enkripsi dan dekripsi [8].

Pada metode ini, pengirim dan penerima harus memiliki kunci rahasia yang sama. Keunggulan utama kriptografi simetris adalah kecepatan proses yang tinggi sehingga cocok digunakan untuk mengenkripsi data berukuran besar seperti dokumen, gambar, video, maupun arsip digital.

Secara umum proses kriptografi simetris dapat digambarkan sebagai berikut:

Plaintext → Enkripsi → *Ciphertext*

Ciphertext → Dekripsi → *Plaintext*

Beberapa algoritma kriptografi simetris yang populer antara lain:

- a. DES (Data Encryption Standard)

- b. Triple DES
- c. Blowfish
- d. Twofish
- e. Advanced encryption Standard (AES)

Saat ini AES menjadi standar enkripsi simetris yang paling banyak digunakan karena memiliki Tingkat keamanan yang tinggi dan performa yang baik [1].

2.3 Advanced Encryption Standard (AES)

Advanced Encryption Standard (AES) merupakan algoritma kriptografi simetris yang ditetapkan oleh National Institute of Standards and Technology (NIST) sebagai standar enkripsi resmi Amerika Serikat [1].

AES dikembangkan berdasarkan algoritma Rijndael yang dirancang oleh Joan Daemen dan Vincent Rijmen. AES bekerja menggunakan blok data berukuran 128-bit dan mendukung tiga ukuran kunci, yaitu:

- a. AES-128
- b. AES-192
- c. AES-256

Semakin panjang ukuran kunci yang digunakan maka semakin besar pula ruang pencarian kunci yang harus ditembus oleh penyerang.

Pada penelitian ini digunakan AES-256 yang memiliki panjang kunci sebesar 256-bit. Dengan ukuran kunci tersebut, jumlah kemungkinan kombinasi kunci mencapai 2^{256} sehingga sangat sulit ditembus menggunakan serangan brute force dengan kemampuan komputasi saat ini [1].

Keunggulan AES antara lain:

- a. Keamanan tinggi.
- b. Efisiensi komputasi.
- c. Dukungan luas pada berbagai platform.
- d. Menjadi standar internasional untuk keamanan data.

2.4 Galois/Counter Mode (GCM)

AES merupakan algoritma *block cipher* yang memerlukan mode operasi untuk mengenkripsi data yang ukurannya melebihi satu blok [9].

Salah satu mode operasi yang paling banyak digunakan saat ini adalah *Galois/Counter Mode* (GCM). Mode ini termasuk *kategori Authenticated Encryption with Associated Data* (AEAD) yang mampu memberikan kerahasiaan sekaligus integritas data [2].

AES-GCM bekerja menggunakan dua mekanisme utama, yaitu [5]:

- a. Counter Mode (CTR) untuk proses enkripsi.
- b. Galois Message Authentication Code (GMAC) untuk autentikasi.

Keunggulan AES-GCM meliputi:

- a. Kecepatan tinggi.
- b. Mendukung pemrosesan paralel.
- c. Menyediakan Authentication Tag.
- d. Mendeteksi perubahan data secara otomatis.

Pada AES-GCM, apabila terdapat perubahan satu bit saja pada *ciphertext*, proses dekripsi akan gagal karena *Authentication Tag* tidak sesuai dengan nilai yang dihasilkan saat enkripsi [2].

Karakteristik tersebut menjadikan AES-GCM sangat cocok digunakan dalam aplikasi keamanan data modern.

2.5 Password-Based Key Derivation Function 2 (PBKDF 2)

Password-Based Key Derivation Function 2 (PBKDF2) merupakan algoritma yang digunakan untuk menghasilkan kunci kriptografi dari password yang dimasukkan pengguna [3].

Password yang dibuat manusia umumnya memiliki tingkat entropi yang rendah sehingga tidak aman digunakan secara langsung sebagai kunci AES. Oleh karena itu diperlukan proses derivasi kunci untuk menghasilkan kunci yang lebih kuat.

PBKDF2 menggunakan tiga komponen utama:

- a. *Password*
- b. *Salt*
- c. Iterasi

Salt merupakan data acak yang ditambahkan ke password sebelum diproses. Penggunaan salt bertujuan mencegah serangan *rainbow table*.

Iterasi digunakan untuk memperlambat proses pembentukan kunci sehingga serangan *brute force* menjadi lebih sulit dilakukan [7].

Pada proyek Secure-Web-Lock digunakan:

- a. SHA-256 sebagai fungsi hash.
- b. Salt sepanjang 16 byte.
- c. Iterasi sebanyak 100.000 kali. Konfigurasi tersebut mampu meningkatkan keamanan kata sandi pengguna secara signifikan.

Konfigurasi tersebut mampu meningkatkan keamanan password pengguna secara signifikan [3].

2.6 Secure Hash Algorithm 256 (SHA-256)

SHA-256 merupakan salah satu anggota keluarga algoritma SHA-2 yang dikembangkan oleh *National Security Agency* (NSA) dan distandardisasi oleh NIST.

SHA-256 menghasilkan nilai hash sepanjang 256-bit yang bersifat unik terhadap data masukan [8].

Karakteristik SHA-256 meliputi:

- a. Deterministik.
- b. Sulit dibalik menjadi data asli.
- c. Tahan terhadap collision attack.
- d. Memiliki tingkat keamanan tinggi.

Dalam penelitian ini SHA-256 digunakan sebagai fungsi hash pada proses PBKDF2 untuk menghasilkan kunci AES yang aman.

2.7 Client-Side Encryption

Client-Side Encryption merupakan metode keamanan data di mana seluruh proses enkripsi dan dekripsi dilakukan pada perangkat pengguna sebelum data dikirimkan ke server [4].

Pada pendekatan ini, server hanya menerima data yang telah terenkripsi sehingga tidak dapat mengetahui isi data pengguna.

Keuntungan *Client-Side Encryption* antara lain:

- a. Privasi lebih tinggi.
- b. Risiko kebocoran data lebih kecil.
- c. Penyedia layanan tidak memiliki akses terhadap isi data.
- d. Mendukung konsep *Zero-Knowledge Architecture*.

Pada aplikasi Secure-Web-Lock seluruh proses kriptografi dilakukan menggunakan Web Crypto API yang berjalan langsung pada browser pengguna.

2.8 Zero-Knowledge Architecture

Zero-Knowledge Architecture merupakan model keamanan yang memastikan penyedia layanan tidak memiliki pengetahuan mengenai data maupun kunci pengguna [6].

Pada arsitektur ini:

- a. Data dienkripsi sebelum meninggalkan perangkat pengguna.
- b. Kunci enkripsi hanya diketahui pengguna.
- c. Server tidak dapat melakukan dekripsi data.

Pendekatan ini banyak digunakan pada layanan penyimpanan data modern karena mampu meningkatkan tingkat privasi pengguna secara signifikan.

Secure-Web-Lock menerapkan konsep *Zero-Knowledge* dengan cara melakukan seluruh proses enkripsi dan dekripsi pada sisi klien.

2.9 Web Crypto API

Web Crypto API merupakan antarmuka kriptografi yang disediakan oleh *browser* modern untuk melakukan operasi keamanan secara langsung pada aplikasi web [4].

Web Crypto API mendukung berbagai fungsi kriptografi seperti:

- a. Pembangkitan bilangan acak.
- b. Enkripsi dan dekripsi.
- c. Derivasi kunci.
- d. *Hashing*.
- e. *Digital signature*.

Keunggulan Web Crypto API dibandingkan implementasi JavaScript murni adalah:

- a. Performa lebih tinggi.
- b. Memanfaatkan implementasi bawaan browser.
- c. Lebih aman.
- d. Mendukung standar kriptografi modern.

Dalam proyek Secure-Web-Lock, Web Crypto API digunakan untuk:

- a. Membuat salt dan initialization vector.
- b. Melakukan PBKDF2.
- c. Menghasilkan kunci AES-256.
- d. Melakukan enkripsi AES-GCM.
- e. Melakukan dekripsi AES-GCM.

2.10 React

React merupakan pustaka JavaScript yang dikembangkan oleh Meta untuk membangun antarmuka pengguna berbasis komponen.

React memungkinkan pengembangan aplikasi web yang responsif, modular, dan mudah dipelihara. Pendekatan berbasis komponen mempermudah proses pengembangan karena setiap bagian antarmuka dapat dibuat secara terpisah dan digunakan kembali.

Pada Secure-Web-Lock, React digunakan untuk membangun seluruh antarmuka aplikasi termasuk halaman enkripsi, dekripsi, indikator keamanan *password*, dan progress monitoring.

2.11 Vite

Vite merupakan build tool modern yang dirancang untuk meningkatkan kecepatan pengembangan aplikasi web.

Keunggulan Vite meliputi:

- a. *Startup* sangat cepat.
- b. *Hot Module Replacement* yang efisien.
- c. Dukungan modern JavaScript.
- d. Optimasi proses build.

Penggunaan Vite pada Secure-Web-Lock membantu mempercepat proses pengembangan dan deployment aplikasi.

2.12 Penelitian Terdahulu

Penelitian mengenai keamanan data berbasis client-side encryption telah banyak dilakukan dalam beberapa tahun terakhir.

Penelitian Das, Sinha, dan Annappa (2019) menunjukkan bahwa implementasi AES-GCM mampu memberikan tingkat keamanan dan performa yang baik pada aplikasi penyimpanan data berbasis cloud [10].

Penelitian Rajan, Mittal, dan Gupta (2025) menyimpulkan bahwa pendekatan client-side encryption mampu mengurangi risiko kebocoran data akibat kompromi server karena kendali kunci enkripsi sepenuhnya berada pada pengguna, bukan pada penyedia layanan cloud [11].

Penelitian Pratama, Yuana, dan Ardinansyah (2026) menunjukkan bahwa penggunaan AES bersama PBKDF2 mampu memberikan ketahanan kriptografis yang baik terhadap serangan brute force tanpa membebani sumber daya perangkat secara signifikan [12].

Berdasarkan penelitian-penelitian tersebut dapat disimpulkan bahwa kombinasi AES-GCM, PBKDF2, dan client-side encryption merupakan solusi yang efektif untuk membangun sistem keamanan data modern berbasis web.

BAB III

ANALISIS DAN PERANCANGAN SISTEM

3.1 Analisis Kebutuhan Sistem

Secure-Web-Lock merupakan aplikasi keamanan data berbasis web yang dirancang untuk melakukan enkripsi dan dekripsi berkas secara lokal pada perangkat pengguna. Sistem ini dikembangkan untuk mengatasi permasalahan keamanan data ketika pengguna ingin menyimpan atau membagikan berkas melalui media internet tanpa harus mempercayakan isi berkas kepada pihak ketiga.

Pendekatan yang digunakan adalah *Client-Side Encryption*, yaitu seluruh proses kriptografi dilakukan pada browser pengguna menggunakan Web Crypto API. Dengan pendekatan ini, berkas asli tidak pernah dikirim ke server sehingga privasi pengguna dapat terjaga secara maksimal.

3.1.1 Kebutuhan Fungsional

- a. Sistem dapat menerima *file* dari pengguna melalui mekanisme *upload* atau *drag and drop*.
- b. Sistem dapat menerima password sebagai kunci enkripsi dan dekripsi.
- c. Sistem dapat menghasilkan kunci AES-256 menggunakan PBKDF2.
- d. Sistem dapat mengenkripsi file menggunakan AES-GCM.
- e. Sistem dapat mendekripsi file menggunakan AES-GCM.
- f. Sistem dapat memverifikasi integritas data menggunakan *Authentication Tag*.
- g. Sistem dapat menampilkan progres proses enkripsi dan dekripsi.
- h. Sistem dapat mengunduh hasil *file* terenkripsi maupun hasil dekripsi.

3.1.2 Kebutuhan Non-Fungsional

- a. Sistem berjalan pada *browser* modern yang mendukung Web Crypto API.
- b. Sistem tidak menyimpan data pengguna pada server.
- c. Sistem mampu mengenkripsi berbagai jenis *file*.
- d. Sistem memiliki antarmuka yang mudah digunakan.
- e. Sistem memiliki tingkat keamanan tinggi melalui AES-GCM dan PBKDF2.

3.2 Analisis Aktor

Analisis aktor dilakukan untuk mengidentifikasi dan mendefinisikan karakteristik serta batasan interaksi dari setiap entitas yang terlibat langsung dalam pengoperasian sistem. Mengingat aplikasi Secure-Web-Lock menerapkan *Zero-Knowledge Architecture* berbasis *Client-Side Encryption* murni, sistem ini dirancang secara independen tanpa memerlukan

manajemen basis data pengguna terpusat ataupun hak akses multi-level seperti administrator peladen.

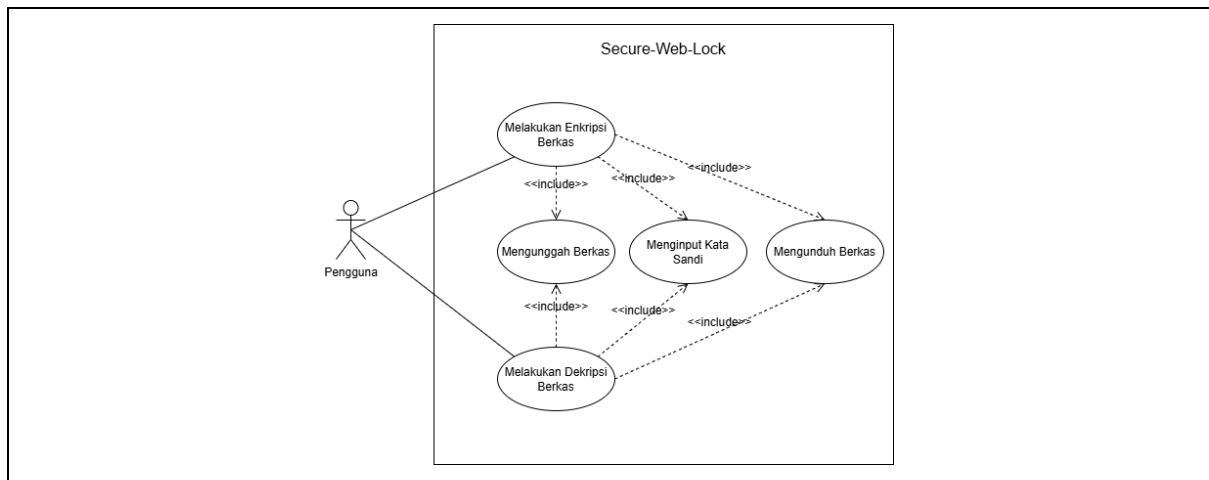
Oleh karena itu, sistem ini hanya memiliki satu aktor utama tunggal yang memegang kendali penuh atas seluruh siklus kriptografi, yaitu Pengguna (*User*). Penjelasan detail mengenai karakteristik, deskripsi peran, serta batasan hak akses dari aktor tersebut dipaparkan pada Tabel 3.1.

Tabel 3.1 Karakteristik dan Hak Akses Aktor Sistem

Nama Aktor	Deskripsi Peran	Hak Akses dan Interaksi Sistem
Pengguna (<i>User</i>)	Individu atau entitas yang memiliki kebutuhan untuk mengamankan kerahasiaan berkas digital atau memulihkan kembali berkas terenkripsi miliknya secara mandiri melalui peramban web.	1. Mengunggah berkas masukan (<i>plaintext</i>) atau berkas terenkripsi ber-ekstensi .enc ke dalam sistem. 2. Menginput kata sandi tekstual yang bertindak sebagai parameter utama pembentukan kunci. 3. Mengeksekusi modul fungsi enkripsi AES-GCM 256-bit dan derivasi kunci PBKDF2. 4. Mengeksekusi modul fungsi dekripsi berkas serta verifikasi nilai <i>Authentication Tag</i>. 5. Mengunduh hasil pemrosesan dokumen kriptografi langsung ke penyimpanan lokal

3.3 Use Case Sistem

Perancangan *use case* bertujuan untuk memetakan batasan sistem (*system boundary*) serta mendefinisikan hubungan interaksi antara aktor dengan fungsionalitas yang disediakan di dalam aplikasi Secure-Web-Lock. Melalui pemodelan ini, seluruh aktivitas digital yang dapat dieksekusi oleh aktor tunggal (Pengguna) di sisi klien dipaparkan secara visual dan naratif.



Gambar 3.1 Diagram Use Case Aplikasi Secure-Web-Lock

Hubungan interaksi antar komponen fungsional dalam arsitektur *client-side encryption* ini digambarkan melalui diagram *use case* yang memisahkan area kerja lokal peramban dengan pengguna. Diagram *use case* yang memisahkan area kerja lokal peramban dengan pengguna. Diagram *use case* untuk aplikasi Secure-Web-Lock ditunjukkan pada Gambar 3.1.

Untuk memperjelas peran dan batasan dari setiap elips fungsionalitas yang terdapat pada Gambar 3.1, deskripsi detail dari masing-masing *use case* dijabarkan pada Tabel 3.2.

Tabel 3.2 Deskripsi Fungsi Use Case Sistem

No	Use Case	Deskripsi
1.	Mengunggah Berkas	Pengguna memasukkan berkas digital ke dalam area kerja aplikasi melalui mekanisme pemilihan dokumen (<i>file picker</i>) atau menyeret berkas (<i>drag-and-drop</i>).
2.	Memasukkan Kata Sandi	Pengguna menginput untaian karakter (<i>string</i>) teks kata sandi pada kolom antarmuka sebagai basis nilai komputasi pembentukan kunci.
3.	Mengenkripsi Berkas	Sistem mentransformasikan data biner asli (<i>plaintext</i>) menjadi data teracak (<i>chipertext</i>) menggunakan Web Crypto API berbasis AES-GCM 256-bit dan PBKDF2.
4.	Mendekripsi Berkas	Sistem memvalidasi integritas biner dokumen dan mengembalikan data teracak kembali ke bentuk format dokumen aslinya.
5.	Mengunduh Berkas Hasil	Sistem secara otomatis memicu fungsi penyimpanan lokal peramban untuk mengunduh dokumen keluaran baik berupa format .enc maupun format awal dokumen.

Sistem ini dirancang dengan aturan ketergantungan relasi <<include>> yang ketat untuk menjamin keamanan dan kelancaran alur pemrosesan biner di dalam memori DOM:

a. Relasi *include* pada operasi kriptografi

Use case "Mengkripsi Berkas" dan "Mendekripsi Berkas" memiliki ketergantungan mutlak terhadap *use case* "Mengunggah Berkas" dan "Memasukkan kata Sandi". Hal ini menandakan bahwa sistem secara struktural memblokir eksekusi tombol operasi apabila objek dokumen belum termuat di RAM atau parameter kata sandi masih kosong.

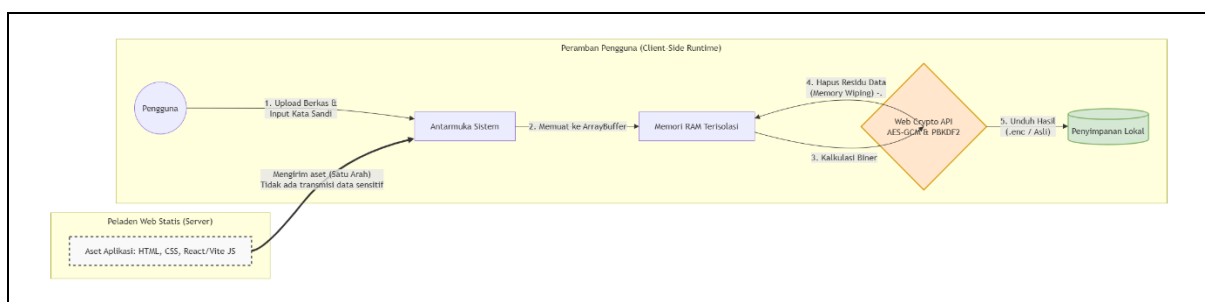
b. Relasi *include* pada operasi untuh

Operasi "Mengkripsi Berkas" dan "Mendekripsi Berkas" secara otomatis mewajibkan sistem untuk memanggil fungsi "Mengunduh Berkas Hasil" sesaat setelah kalkulasi matematika Web Crypto API dinyatakan selesai dan valid. Hal ini meminimalkan durasi pengendapan data sensitif di dalam penyangga (*buffer*) memori perangkat.

3.4 Perancangan Arsitektur Sistem

Perancangan arsitektur pada aplikasi Secure-Web-Lock didasarkan pada model *Zero-Knowledge Architecture* yang mengimplementasikan metode *Client-Side Encryption* secara murni. Arsitektur ini dirancang untuk memastikan bahwa seluruh data sensitif milik pengguna, baik berupa berkas asli (*plaintext*) maupun kata sandi, tetap berada dalam kendali penuh perangkat klien dan tidak pernah terekspos ke jaringan luar.

Seperti yang diilustrasikan pada diagram arsitektur sistem (Gambar 3.2), topologi ini memisahkan entitas peladen (*server*) dengan lingkungan kerja peramban (*browser*) pengguna secara tegas:



Gambar 3.2 Rancangan Arsitektur Sistem

a. Peladen web statis (*static web server*)

Pada arsitektur ini, peladen tidak memiliki peran dalam pengolahan data biner ataupun penyimpanan basis data pengguna. Peladen hanya bertugas menyajikan dan mengirimkan aset statis aplikasi (berkas HTML, CSS, dan JavaScript dari React/Vite)

secara satu arah (*one-way*) ke perangkat pengguna saat aplikasi pertama kali diakses. Tidak ada jalur transmisi balasan dari peramban ke peladen yang membawa parameter kata sandi maupun dokumen.

b. Perambang pengguna (*client-side runtime*)

Setelah aset aplikasi termuat, peramban mengambil alih seluruh siklus kerja aplikasi sebagai lingkungan eksekusi yang mandiri dan terisolasi. Alur pemrosesan data di dalam klien terjadi melalui tahapan berikut:

- i. Interaksi Pengguna: Pengguna mengunggah berkas dan menginput parameter kata sandi melalui Antarmuka Sistem (UI).
- ii. Alokasi Memori: Sistem pada antarmuka menangkap berkas tersebut dan memuatnya ke dalam bentuk *ArrayBuffer* di dalam ruang memori acak (RAM) peramban yang terisolasi.
- iii. Kalkulasi Kriptografi: Biner berkas dan kata sandi diteruskan ke dalam mesin Web Crypto API. Mesin ini memanfaatkan akselerasi perangkat keras lokal untuk mengeksekusi fungsi derivasi kunci PBKDF2 dan operasi inti enkripsi/dekripsi AES-GCM 256-bit.
- iv. Pembersihan Residu (*Memory Wiping*): Sesaat setelah kalkulasi matematis selesai, mesin kriptografi akan langsung menghapus sisa data sensitif dari penyangga (*buffer*) RAM untuk memitigasi risiko pencurian sisa memori.
- v. Penyimpanan Lokal: Hasil akhir pemrosesan, baik berupa berkas terenkripsi (.enc) maupun berkas terdekripsi, secara otomatis diekspor dan diunduh secara langsung ke dalam penyimpanan fisik lokal perangkat pengguna tanpa intervensi pihak ketiga.

3.5 Perancangan Proses Enkripsi

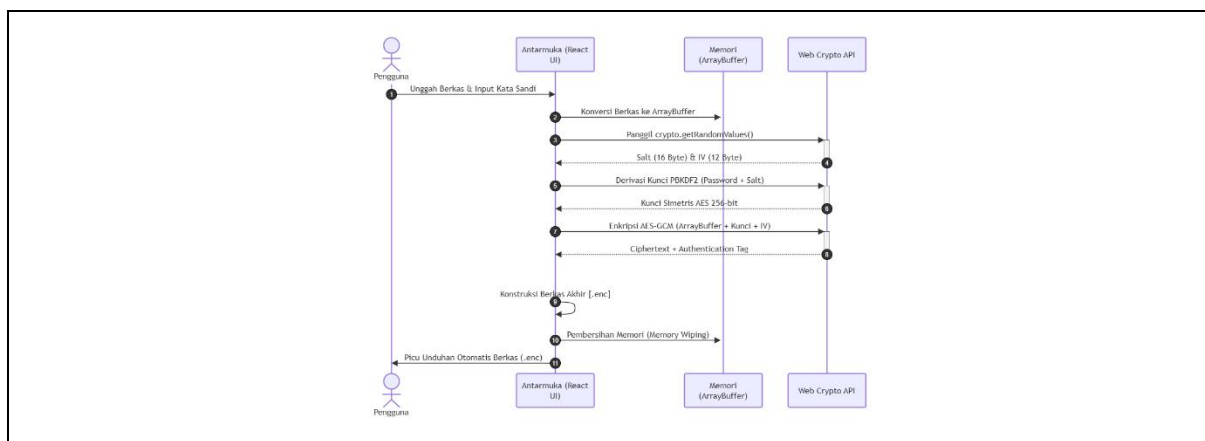
Perancangan proses enkripsi menjelaskan alur transformasi data secara kronologis, mulai dari saat pengguna memasukkan berkas asli (*plaintext*) hingga sistem menghasilkan berkas sandi terenkripsi berformat .enc. Seluruh rangkaian proses ini dirancang berjalan secara asinkron di dalam memori lokal peramban tanpa melibatkan lalu lintas jaringan peladen.

Secara sistematis, tahapan komputasi biner pada modul enkripsi Secure-Web-Lock dijabarkan sebagai berikut:

- a. Pemuatan Dokumen dan Kata Sandi: Pengguna mengunggah dokumen asli dan memasukkan untaian kata sandi ke antarmuka aplikasi.

- b. Konversi ArrayBuffer: Sistem membaca struktur fisik berkas menggunakan objek `FileReader` secara asinkron dan mengonversinya menjadi struktur data biner `ArrayBuffer` di dalam RAM perangkat klien.
- c. Pembangkitan Parameter Acak (CSPRNG): Mesin peramban memanggil fungsi `crypto.getRandomValues()` untuk membangkitkan dua komponen acak aman secara kriptografis, yaitu nilai *Salt* sepanjang 16 byte dan *Initialization Vector* (IV) sepanjang 12 byte.
- d. Derivasi Kunci Kriptografi: Teks kata sandi pengguna dikombinasikan dengan nilai *Salt* yang baru dibentuk, kemudian diproses melalui fungsi PBKDF2 menggunakan fungsi dasar hash SHA-256 dengan total pengulangan sebanyak 100.000 iterasi untuk menghasilkan kunci simetris AES 256-bit.
- e. Eksekusi Enkripsi AES-GCM: Data `ArrayBuffer` berkas asli dienkripsi menggunakan kunci simetris AES 256-bit dan IV 12 byte melalui Web Crypto API. Proses ini menghasilkan blok data teracak (*ciphertext*) sekaligus membentuk komponen *Authentication Tag* secara otomatis pada akhir blok.
- f. Konstruksi Berkas Akhir: Sistem menggabungkan seluruh komponen biner yang terpisah ke dalam satu objek `Blob` tunggal dengan susunan format linier terstruktur: `[Salt (16 byte)] + [IV (12 byte)] + [Ciphertext + Authentication Tag]`.
- g. Pembersihan Memori dan Unduhan: Fungsi `memory wiping` dijalankan untuk membersihkan sisa variabel penyangga di dalam RAM klien. Selanjutnya, sistem memicu pengunduhan berkas keluaran ber-ekstensi `.enc` ke media penyimpanan pengguna.

Alur interaksi komponen dan urutan pemanggilan fungsi kriptografi selama proses enkripsi digambarkan melalui diagram sekuensial pada Gambar 3.3.



Gambar 3.3 Diagram Sekuensial Proses Enkripsi Berkas pada Aplikasi Secure-Web-Lock

Untuk memastikan portabilitas berkas agar dapat didekripsi kembali di masa mendatang, format berkas luaran .enc tidak hanya berisi data teracak, melainkan wajib membawa parameter metadata pembentuk kunci. Desain anatomi susunan biner berkas hasil enkripsi ditunjukkan pada Tabel 3.3.

Tabel 3.3 Spesifikasi Pemetaan Biner Berkas Format

Segmen Biner	Ukuran Data	Deskripsi Fungsi
Header Segmen 1 (<i>Salt</i>)	16 Byte	Nilai acak unik sebagai parameter input PBKDF2 untuk mencegah serangan <i>rainbow table</i> saat rekonstruksi kunci.
Header Segmen 2 (<i>Inisialisasi Vector</i>)	12 Byte	Nilai acak penambah entropi biner untuk memastikan enkripsi blok AES-GCM tidak menghasilkan pola biner yang sama.
Payload Segmen 3 (<i>Chiphertext</i>)	Dinamis (Sesuai ukuran awal)	Inti data berkas asli yang telah diacak secara matematis melalui algoritma AES 256-bit.
Footer Segmen 4 (<i>Authentication Tag</i>)	16 Byte (Otomatis di akhir data)	Kode autentikasi biner Galois untuk memvalidasi integritas berkas dari ancaman manipulasi (<i>anti-tampering</i>).

3.6 Perancangan Proses Dekripsi

Perancangan proses dekripsi merupakan kebalikan dari proses enkripsi, namun dilengkapi dengan mekanisme validasi integritas biner yang ketat. Proses ini bertujuan untuk memeriksa keaslian dokumen terenkripsi (.enc), memastikan bahwa data belum mengalami modifikasi, dan mengembalikannya ke bentuk format dokumen aslinya menggunakan Web Crypto API.

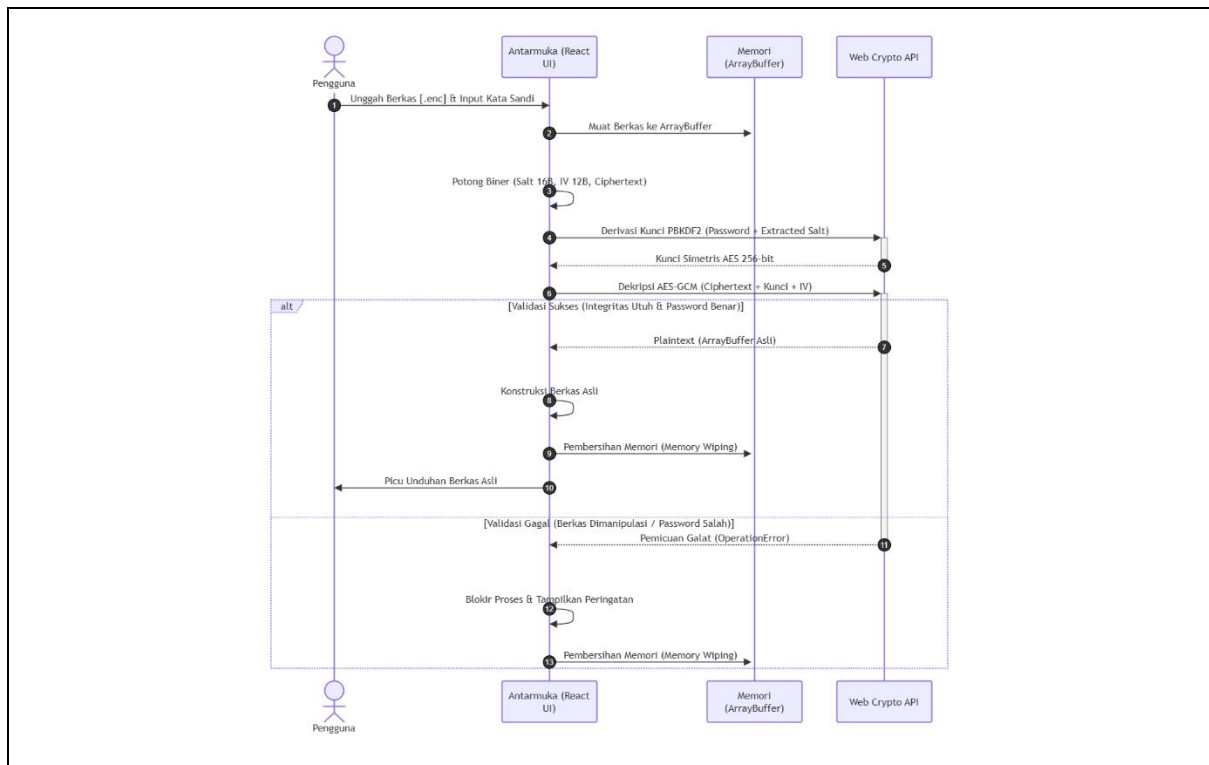
Secara kronologis, tahapan rekonstruksi data pada modul dekripsi Secure-Web-Lock dijabarkan sebagai berikut:

- Pemuatan Berkas dan Kata Sandi:** Pengguna mengunggah berkas terenkripsi berformat .enc dan memasukkan kata sandi yang digunakan saat proses enkripsi awal.
- Pembacaan dan Pemotongan Biner (Slicing):** Sistem membaca berkas .enc ke dalam bentuk ArrayBuffer di dalam memori RAM klien. Selanjutnya, sistem melakukan

pemotongan segmen biner secara presisi berdasarkan aturan format yang telah ditentukan:

- i. 16 byte pertama diisolasi dan diekstrak sebagai nilai *Salt*.
 - ii. 12 byte berikutnya diisolasi dan diekstrak sebagai nilai *Initialization Vector* (IV).
 - iii. Sisa byte ke belakang diidentifikasi sebagai data teracak (*ciphertext*) yang berpasangan dengan Authentication Tag.
- c. Rekonstruksi Kunci Kriptografi: Kata sandi tekstual yang diinput oleh pengguna dikombinasikan dengan nilai *Salt* hasil ekstrak berkas, kemudian diproses ulang melalui fungsi PBKDF2 (SHA-256, 100.000 iterasi) untuk menghasilkan kembali kunci simetris AES 256-bit yang identik.
- d. Dekripsi dan Validasi Integritas: Kunci simetris, IV, dan ciphertext dikirim ke fungsi dekripsi Web Crypto API. Komponen *Galois Message Authentication Code* (GMAC) pada mesin peramban secara otomatis akan melakukan kalkulasi matematika runtime untuk mencocokkan nilai *Authentication Tag* yang berada di dalam berkas.
- e. Penanganan hasil validasi:
- i. Jika Validasi Sukses: Data asli (*plaintext*) berhasil dikembalikan menjadi dokumen utuh, sistem memicu pengunduhan otomatis ke format aslinya, dan menghapus variabel sensitif dari RAM melalui fungsi *memory wiping*.
 - ii. Jika Validasi Gagal: Apabila kata sandi salah atau struktur biner berkas telah mengalami manipulasi sekecil satu bit saja (*tampering*), peramban secara otomatis akan memicu kegagalan sistem (*OperationError*). Sistem akan menangkap *exception* tersebut, memblokir rekonstruksi berkas secara absolut, menampilkan notifikasi peringatan di antarmuka, dan langsung membersihkan RAM.

Alur interaksi komponen, percabangan kondisi validasi, serta urutan pemanggilan fungsi selama proses dekripsi digambarkan melalui diagram sekuensial pada Gambar 3.4.



Gambar 3.4 Diagram Sekuensial Proses Dekripsi Berkas pada Aplikasi Secure-Web-Lock

BAB IV

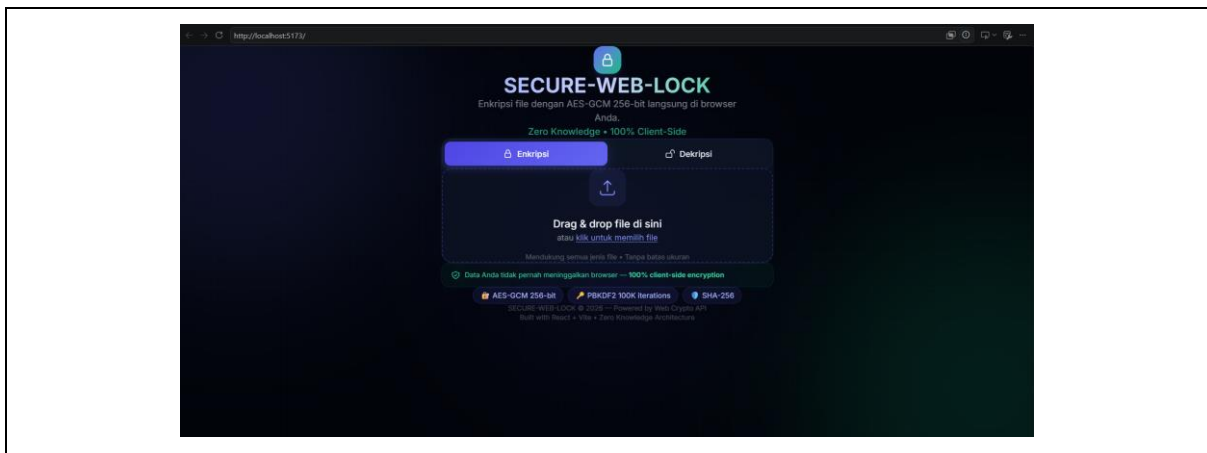
IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Sistem

Implementasi aplikasi Secure-Web-Lock dilakukan dengan mentransformasikan seluruh rancangan sistem ke dalam kode program berbasis web yang dijalankan sepenuhnya pada lingkungan sisi klien (*client-side runtime*). Pembahasan mengenai detail implementasi dari masing-masing komponen perangkat lunak dipaparkan pada sub-bab berikut.

4.1.1 Lingkungan pengembangan dan Modul Unggah Berkas

Antarmuka utama aplikasi beserta modul unggah berkas dirancang untuk memfasilitasi interaksi pengguna di sisi klien. Hasil implementasi antarmuka ini dapat dilihat pada Gambar 4.1.



Gambar 4.1 Implementasi Antarmuka Utama Aplikasi dan modul Unggah Berkas Menggunakan React dan Tailwind CSS

Aplikasi Secure-Web-Lock berhasil dibangun sebagai sebuah *Single Page Application* (SPA) menggunakan pustaka komponen React dan Vite sebagai *build tool* pemrosesnya. Untuk menghasilkan pengalaman pengguna yang optimal, antarmuka pengguna (UI) dikembangkan dengan memanfaatkan kerangka kerja *Tailwind CSS*. Pada bagian tengah antarmuka, modul unggah berkas bertindak sebagai gerbang utama masuknya dokumen yang akan diproses di dalam memori lokal peramban. Modul ini menyediakan dua mekanisme interaksi yang intuitif, yaitu melalui metode penekanan tombol pemilih dokumen konvensional (*file picker*) maupun fungsi penyeretan berkas secara langsung (*drag-and-drop*).

4.1.2 Implementasi *Password Strength Meter*

Sistem dilengkapi dengan indikator kekuatan kata sandi yang terintegrasi langsung pada area input formulir, seperti yang ditunjukkan pada Gambar 4.2.



Gambar 4.2 Implementasi Indikator Kekuatan Kata Sandi Secara *Real-Time* pada Antarmuka Sistem

Guna memitigasi risiko keamanan akibat kecenderungan pengguna dalam memilih kata sandi yang lemah, fitur *Password Strength Meter* ini dievaluasi secara *real-time* setiap kali pengguna mengetikkan karakter. Indikator kekuatan ini memvalidasi kompleksitas untaian karakter kata sandi berdasarkan panjang total karakter, kombinasi huruf besar-kecil, kemunculan angka, serta keberadaan karakter khusus. Umpan balik visual ini memaksa pengguna untuk memenuhi standar keamanan minimum sebelum kata sandi diproses oleh mesin kriptografi.

4.1.3 Implementasi Pembentukan Kunci PBKDF2

Fungsi pembentukan kunci ini diimplementasikan murni menggunakan metode bawaan peramban (*Web Crypto API*) tanpa mengandalkan pustaka pihak ketiga eksternal. Modul ini bertugas mentransformasikan string pengguna menjadi kunci simetris AES berukuran 256-bit. Parameter keamanan yang ditanamkan secara statis ke dalam sistem meliputi penggunaan fungsi *hash* SHA-256, *Salt* sepanjang 16 *byte*, dan nilai iterasi pengulangan sebesar 100.000 kali. Besaran iterasi ini ditetapkan secara spesifik untuk memberikan perlindungan matematis yang kuat guna menahan laju ancaman serangan *brute force*.

Berikut adalah potongan kodenya:

```
async function deriveKey(password, salt, usage) {
  const passwordBuffer = new TextEncoder().encode(password);
  const baseKey = await window.crypto.subtle.importKey(
    'raw', passwordBuffer, 'PBKDF2', false, ['deriveKey']
  );
  return window.crypto.subtle.deriveKey(
    { name: 'PBKDF2', salt, iterations: PBKDF2_ITERATIONS, hash: 'SHA-256' },
    baseKey,
```



```
{ name: 'AES-GCM', length: 256 },  
  false,  
  [usage]  
);  
}
```

4.1.4 Implementasi Kriptografi AES-GCM

Algoritma *Advanced Encryption Standard* dengan mode operasi *Galois/Counter Mode* (AES-GCM) 256-bit diimplementasikan sebagai fondasi pengamanan berkas. Pemanggilan fungsi ini dikendalikan langsung oleh *Web Crypto API* untuk menegakkan aspek kerahasiaan dan integritas data secara simultan. Melalui implementasi mode GCM, peramban secara otomatis akan menghitung dan menyematkan komponen *Authentication Tag* di akhir data biner. Hal ini memberikan sistem kemampuan perlindungan *anti-tampering* yang absolut, di mana proses dekripsi akan langsung diblokir (memunculkan *OperationError*) jika struktur biner berkas mengalami manipulasi. Berikut adalah potongan kodenya:

```
// Eksekusi Enkripsi AES-GCM (Auth Tag otomatis terlampir di akhir)  
const ciphertext = await window.crypto.subtle.encrypt(  
  { name: 'AES-GCM', iv },  
  key,  
  fileBuffer  
);  
  
// Eksekusi Dekripsi AES-GCM (Termasuk proses validasi integritas / anti-  
tampering)  
const plaintext = await window.crypto.subtle.decrypt(  
  { name: 'AES-GCM', iv },  
  key,  
  ciphertext  
);
```

4.1.5 Implementasi Mekanisme *Memory Wiping*

Implementasi mekanisme *memory wiping* otomatis ini bertujuan menutup celah kerentanan kebocoran informasi pasca-pemrosesan melalui metode pembongkaran memori fisik (*memory dumping analysis*). Sesaat setelah operasi kriptografi selesai dan berkas luaran berhasil diunduh oleh pengguna, sistem akan langsung menimpa (*overwrite*) seluruh variabel penampung data sensitif seperti dokumen asli dan kunci rahasia di dalam *ArrayBuffer* menggunakan nilai acak (*random values*). Mekanisme ini secara efektif memastikan ketiadaan residu jejak digital di dalam memori peramban klien. Berikut adalah potongan kodenya:

```

/**
 * Attempt to wipe an ArrayBuffer from memory.
 */
function wipeBuffer(buffer) {
  try {
    const view = new Uint8Array(buffer);
    // Menimpa (overwrite) data sensitif di RAM dengan nilai acak
    window.crypto.getRandomValues(view);
  } catch {
    // Buffer may already be detached (transferred), which is fine
  }
}

```

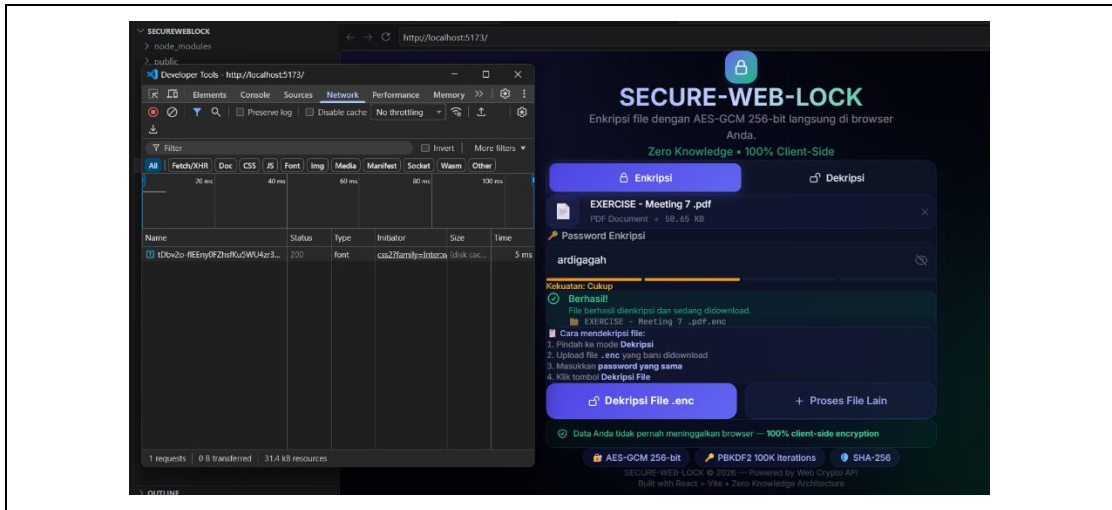
4.2 Pengujian Fungsional

Pengujian fungsional pada aplikasi Secure-Web-Lock dilakukan menggunakan metode *Black Box Testing*. Pendekatan ini berfokus pada evaluasi input dan output sistem tanpa perlu menginspeksi struktur kode internal, guna memvalidasi apakah seluruh fungsionalitas kriptografi berjalan sesuai dengan spesifikasi arsitektur *Zero-Knowledge* yang telah dirancang. Pengujian dibagi ke dalam tiga skenario utama yang menyorot pada aspek privasi transmisi, integritas berkas, dan manajemen memori.

a. Skenario 1: uji arsitektur *zero-knowledge* (privasi transmisi)

Pengujian ini bertujuan untuk memvalidasi ketiadaan transmisi data sensitif ke jaringan luar selama proses kriptografi berlangsung.

- i. Langkah Pengujian: Mengunggah dokumen dan mengeksekusi proses enkripsi/dekripsi sembari memonitor lalu lintas HTTP/HTTPS melalui fasilitas *Network tab* pada *Developer Tools* peramban klien.
- ii. Hasil yang Diharapkan: Indikator jaringan tidak mendeteksi adanya permintaan (*request*) berbasis POST atau PUT yang membawa *payload* berisi parameter kata sandi maupun biner dokumen menuju peladen (*server*) mana pun.
- iii. Hasil Pengujian: Sesuai harapan. Seperti yang ditunjukkan pada Gambar 4.3, seluruh lalu lintas jaringan terpantau statis dan bersih dari transmisi data keluar saat proses enkripsi berjalan, membuktikan penerapan privasi yang absolut.

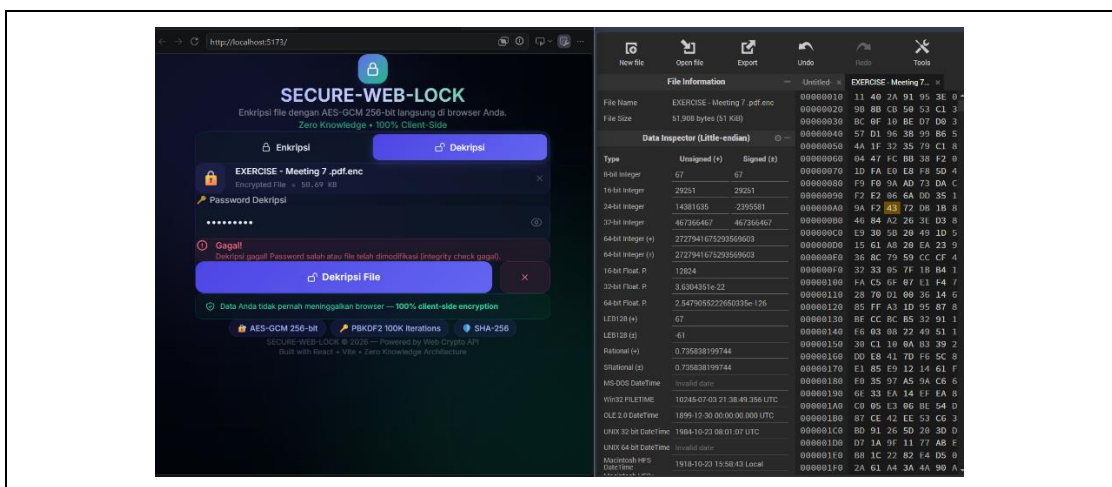


Gambar 4.3 Hasil Pemantauan Lalu Lintas Jaringan Pada Tab *Network* yang Membuktikan Ketiadaan Transmisi Data

b. Skenario 2: uji integritasi *anti-tampering* (manipulasi berkas)

Pengujian ini ditujukan untuk menguji ketahanan algoritma AES-GCM terhadap upaya modifikasi atau sabotase biner berkas secara ilegal.

- Langkah Pengujian: Memodifikasi satu *byte* heksadesimal secara acak pada berkas .enc yang valid menggunakan aplikasi *Hex Editor*, lalu mengunggahnya kembali ke sistem untuk didekripsi menggunakan kata sandi yang benar.
- Hasil yang Diharapkan: Komponen *Galois Message Authentication Code* (GMAC) memicu penolakan otomatis (*OperationError*) akibat ketidaksesuaian nilai *Authentication Tag*, dan sistem memblokir unduhan berkas.
- Hasil Pengujian: Sesuai harapan. Gambar 4.4 menunjukkan sistem secara mutlak menggagalkan proses dekripsi pada berkas yang telah dimanipulasi binernya, memberikan notifikasi kegagalan integritas kepada pengguna.

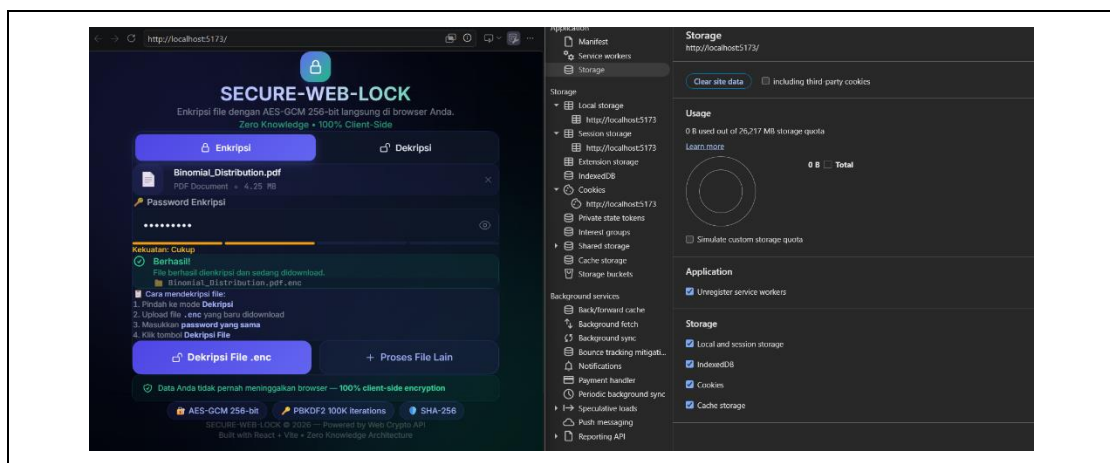


Gambar 4.4 Penolakan Sistem Terhadap Upaya Dekripsi berkas yang Telah Dimanipulasi Melalui *Hex Editor*

c. Skenario 3: uji residu memori (*memory wiping*)

Pengujian ini dilakukan untuk memastikan tidak adanya variabel sensitif yang tertinggal di dalam penyangga peramban setelah sesi kerja berakhir.

- i. Langkah Pengujian: Menginspeksi fasilitas penyimpanan lokal peramban (*Local Storage*, *Session Storage*, dan *Cookies*) melalui fasilitas *Application tab* segera setelah berkas hasil kriptografi diunduh.
- ii. Hasil yang Diharapkan: Tidak ditemukan adanya jejak rekam (*log*), parameter Salt, kunci AES, maupun string kata sandi di dalam fasilitas penyimpanan tersebut.
- iii. Hasil Pengujian: Sesuai harapan. Berdasarkan inspeksi pada Gambar 4.5, fungsi *memory wiping* bekerja secara optimal dengan menimpa sisa penyangga, terbukti dari indikator kapasitas penyimpanan yang menunjukkan nilai 0 Byte pasca-operasi.



Gambar 4.5 Inspeksi Fasilitas *Storage* Peramban yang Membuktikan Ketiadaan Residu Memori Setelah Proses Enkripsi Selesai

4.3 Analisis Hasil pengujian

Berdasarkan ketiga skenario *Black Box Testing* yang telah dieksekusi, aplikasi Secure-Web-Lock dinyatakan berhasil memenuhi seluruh kriteria fungsionalitas dan standar keamanan yang ditetapkan. Integrasi algoritma AES-GCM 256-bit dan PBKDF2 melalui antarmuka *Web Crypto API* terbukti mampu menggeser seluruh beban komputasi secara eksklusif ke sisi klien (*Client-Side Encryption*). Keberhasilan Skenario 1 mengonfirmasi penerapan prinsip privasi absolut, Skenario 2 membuktikan perlindungan kokoh terhadap bahaya korupsi data (*data corruption*), dan Skenario 3 menegaskan mitigasi yang efektif terhadap ancaman pencurian memori (seperti *Cross-Site Scripting* atau eksploitasi akses fisik). Secara keseluruhan, sistem

ini beroperasi secara mandiri, stabil, dan aman tanpa memiliki ketergantungan operasional terhadap peladen pihak ketiga.

BAB 5

PENUTUP

5.1 Kesimpulan

Berdasarkan seluruh tahapan analisis, perancangan, implementasi, serta serangkaian skenario pengujian yang telah dieksekusi pada aplikasi Secure-Web-Lock, maka dapat ditarik beberapa kesimpulan sebagai berikut:

- a. Aplikasi Secure-Web-Lock berhasil dirancang dan dibangun sebagai perangkat lunak pengamanan dokumen berbasis *Single Page Application* (SPA) menggunakan kerangka kerja React, Vite, dan Tailwind CSS, di mana seluruh pemrosesan data biner berjalan murni di lingkungan peramban klien (*client-side runtime*).
- b. Implementasi algoritma *Advanced Encryption Standard* dengan mode operasi *Galois/Counter Mode* (AES-GCM) 256-bit terbukti efektif dalam menjaga kerahasiaan (*confidentiality*) sekaligus menegaskan aspek integritas (*integrity*) berkas digital. Mekanisme *Authentication Tag* sepanjang 16 *byte* bawaan mode GCM berhasil memberikan perlindungan *anti-tampering* absolut, di mana sistem secara otomatis memblokir proses dekripsi jika terdeteksi adanya manipulasi biner berkas sekecil satu bit pada *Hex Editor*.
- c. Algoritma *Password-Based Key Derivation Function 2* (PBKDF2) yang dikombinasikan dengan fungsi *hash* SHA-256 dan konfigurasi statis 100.000 kali iterasi terbukti berhasil merekonstruksi kunci simetris 256-bit yang kuat dari kata sandi tekstual pengguna. Tingginya jumlah iterasi memberikan jeda tunda (*delay*) komputasi yang aman di sisi klien untuk memperkuat ketahanan aplikasi dari ancaman serangan *brute force* dan *dictionary attack*.
- d. Penerapan konsep *Zero-Knowledge Architecture* telah tervalidasi secara empiris melalui pengujian *Black Box*. Hasil inspeksi lalu lintas jaringan (*Network tab*) membuktikan tidak ada satu pun paket data sensitif (baik berkas asli maupun kata sandi) yang ditransmisikan keluar ke peladen pihak ketiga. Selain itu, pengujian residu memori mengonfirmasi bahwa fungsi *memory wiping* bekerja optimal menimpa penyangga RAM menggunakan nilai acak sesaat setelah dokumen berhasil diunduh, sehingga memitigasi risiko kerentanan *memory dumping*.

5.2 Saran

Meskipun sistem telah memenuhi standar fungsionalitas keamanan yang ditargetkan, terdapat beberapa keterbatasan teknis yang dapat dijadikan acuan pengembangan bagi penelitian selanjutnya, antara lain:

- a. Implementasi Streams API: Pada penelitian ini, berkas dibaca secara utuh ke dalam ArrayBuffer memori RAM klien, sehingga berisiko menimbulkan galat *out-of-memory* pada peramban jika memproses berkas berskala raksasa (ukuran *Gigabyte*). Penelitian berikutnya disarankan menerapkan mekanisme *streaming* data biner agar pemrosesan berkas dapat dilakukan secara bertahap dan efisien.
- b. Penambahan Fitur *Asymmetric Key Sharing*: Sistem saat ini mengharuskan pengirim dan penerima berkas berbagi kata sandi yang sama secara manual (kriptografi simetris). Pengembangan selanjutnya dapat mengintegrasikan algoritma kunci publik (seperti RSA atau ECC) untuk memfasilitasi distribusi kunci enkripsi antar-pengguna secara aman tanpa merusak esensi arsitektur *client-side encryption*.
- c. Optimasi *Multi-Threading* via Web Workers: Untuk mencegah terjadinya pembekuan sesaat (*freezing*) pada antarmuka pengguna (UI) peramban ketika mengolah berkas berukuran besar, disarankan untuk menggeser beban komputasi Web Crypto API ke dalam utas latar belakang menggunakan fasilitas *Web Workers*.

DAFTAR PUSTAKA

- [1] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS PUB 197 Update 1, Gaithersburg, MD, USA, 2023.
- [2] National Institute of Standards and Technology, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, Gaithersburg, MD, USA.
- [3] National Institute of Standards and Technology, *Recommendation for Password-Based Key Derivation*, NIST SP 800-132, Gaithersburg, MD, USA.
- [4] Mozilla Developer Network, "Web Crypto API," MDN Web Docs, 2025. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API. Accessed: Jun. 3, 2026.
- [5] Mozilla Developer Network, "AesGcmParams," MDN Web Docs, 2025. Available: <https://developer.mozilla.org/en-US/docs/Web/API/AesGcmParams>. Accessed: Jun. 3, 2026.
- [6] OWASP Foundation, *Cryptographic Storage Cheat Sheet*, 2024.
- [7] OWASP Foundation, *Password Storage Cheat Sheet*, 2025.
- [8] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 9th ed. Pearson, 2024.
- [9] C. Paar and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*, Springer, 2023.
- [10] P. K. Das, N. Sinha, and Annappa B., "Data Privacy Preservation using AES-GCM Encryption in Heroku Cloud," *International Journal of Recent Technology and Engineering (IJRTE)*, vol. 8, no. 3, pp. 7544–7548, Sep. 2019.
- [11] G. V. Rajan, R. Mittal, and A. Gupta, "Secured Cloud Storage with Client Side Encryption," *SSRN Electronic Journal*, Dec. 2025.
- [12] M. W. R. Pratama, H. Yuana, and F. T. Ardinansyah, "Development and Evaluation of a Desktop Academic Data Encryption Application Using AES with Password-Based Key Management," *Journal of Artificial Intelligence and Engineering Applications (JAIEA)*, vol. 5, no. 3, pp. 4197–4210, Jun. 2026.
- [13] Verizon Business, *2025 Data Breach Investigations Report (DBIR)*, Basking Ridge, NJ, USA, 2025.
- [14] Identity Theft Resource Center, *2025 Annual Data Breach Report*, San Diego, CA, USA, Jan. 2026.

LAMPIRAN

Daftar kode yang dilampirkan:

No.	Nama File	Fungsi Utama
A.1	src/hooks/useCrypto.js	Berisi logika kriptografi utama (core cryptographic logic) yang menggunakan Web Crypto API secara asinkron.
A.2	src/utils/helpers.js	Berisi fungsi helper utilitas untuk penanganan file dan manipulasi data.
A.3	src/App.jsx	Komponen utama (entri utama UI/UX) dari aplikasi React.
A.4	src/components/Dropzone.jsx	Komponen area seret dan lepas (drag & drop) file.
A.5	src/components/PasswordField.jsx	Komponen kolom input kata sandi yang disesuaikan (custom password field).
A.6	src/components/FileInfo.jsx	Komponen yang menampilkan informasi mendetail tentang file yang telah berhasil dimuat oleh dropzone.

A.1 Kode Program src/hooks/useCrypto.js

Deskripsi: Berisi logika kriptografi utama (core cryptographic logic) yang menggunakan Web Crypto API secara asinkron. Berfungsi untuk melakukan penurunan kunci enkripsi AES-GCM 256-bit melalui PBKDF2 (dengan 100.000 iterasi dan salt 16-byte), melakukan proses enkripsi (menggabungkan Salt + IV + Ciphertext), serta proses dekripsi file dengan verifikasi integritas data (integrity check). File ini juga dilengkapi fungsi memory wiping untuk menghapus ArrayBuffer sensitif dari memori browser setelah selesai diproses.

```
/**
 * useCrypto.js - Core cryptographic logic using Web Crypto API.
 * Implements AES-GCM 256-bit encryption with PBKDF2 key derivation.
 *
 * File format: [Salt (16 bytes)] + [IV (12 bytes)] + [Ciphertext + GCM
Auth Tag]
 */
import { useState, useCallback, useRef } from 'react';

const SALT_LENGTH = 16;
const IV_LENGTH = 12;
const PBKDF2_ITERATIONS = 100000;

/**
 * Derive an AES-GCM 256-bit key from a password and salt using PBKDF2.
 */
async function deriveKey(password, salt, usage) {
  const passwordBuffer = new TextEncoder().encode(password);
  const baseKey = await window.crypto.subtle.importKey(
    'raw', passwordBuffer, 'PBKDF2', false, ['deriveKey']
  );
  return window.crypto.subtle.deriveKey(
    { name: 'PBKDF2', salt, iterations: PBKDF2_ITERATIONS, hash: 'SHA-
256' },
    baseKey,
    { name: 'AES-GCM', length: 256 },
    false,
    [usage]
  );
}
```

```

/**
 * Custom hook providing encrypt/decrypt functionality with progress
tracking.
 */
export function useCrypto() {
  const [status, setStatus] = useState('idle'); // idle | processing |
success | error
  const [progress, setProgress] = useState(0);
  const [error, setError] = useState(null);
  const [resultBlob, setResultBlob] = useState(null);
  const abortRef = useRef(false);

  const reset = useCallback(() => {
    setStatus('idle');
    setProgress(0);
    setError(null);
    if (resultBlob) {
      setResultBlob(null);
    }
    abortRef.current = false;
  }, [resultBlob]);

  const abort = useCallback(() => {
    abortRef.current = true;
  }, []);

  /**
   * Encrypt a file with the given password.
   * @param {File} file
   * @param {string} password
   * @returns {Promise<Blob|null>}
   */
  const encrypt = useCallback(async (file, password) => {
    try {
      abortRef.current = false;
      setStatus('processing');
      setError(null);
      setProgress(5);

      // Step 1: Generate random Salt and IV

```

```

        const salt = window.crypto.getRandomValues(new
Uint8Array(SALT_LENGTH));
        const iv = window.crypto.getRandomValues(new
Uint8Array(IV_LENGTH));
        setProgress(10);

        // Step 2: Derive key from password
        const key = await deriveKey(password, salt, 'encrypt');
        setProgress(25);

        if (abortRef.current) { reset(); return null; }

        // Step 3: Read the file
        const fileBuffer = await file.arrayBuffer();
        setProgress(50);

        if (abortRef.current) { reset(); return null; }

        // Step 4: Encrypt with AES-GCM (Auth Tag auto-appended)
        const ciphertext = await window.crypto.subtle.encrypt(
            { name: 'AES-GCM', iv },
            key,
            fileBuffer
        );
        setProgress(85);

        // Step 5: Combine Salt + IV + Ciphertext into final blob
        const blob = new Blob([salt, iv, ciphertext]);
        setProgress(100);
        setResultBlob(blob);
        setStatus('success');

        // Auto-wipe sensitive data from memory
        wipeBuffer(fileBuffer);

        return blob;
    } catch (err) {
        console.error('Encryption error:', err);
        setError('Gagal mengenkripsi file. Silakan coba lagi.');
```

```

    }
}, [reset]));

/**
 * Decrypt an encrypted file with the given password.
 * @param {File} file
 * @param {string} password
 * @returns {Promise<Blob|null>}
 */
const decrypt = useCallback(async (file, password) => {
  try {
    abortRef.current = false;
    setStatus('processing');
    setError(null);
    setProgress(5);

    // Step 1: Read the encrypted file
    const fileBuffer = await file.arrayBuffer();
    setProgress(15);

    // Step 2: Extract Salt, IV, and Ciphertext
    const salt = new Uint8Array(fileBuffer.slice(0, SALT_LENGTH));
    const iv = new Uint8Array(fileBuffer.slice(SALT_LENGTH,
SALT_LENGTH + IV_LENGTH));
    const ciphertext = fileBuffer.slice(SALT_LENGTH + IV_LENGTH);
    setProgress(25);

    if (abortRef.current) { reset(); return null; }

    // Step 3: Derive key from password
    const key = await deriveKey(password, salt, 'decrypt');
    setProgress(50);

    if (abortRef.current) { reset(); return null; }

    // Step 4: Decrypt with AES-GCM (integrity check included)
    const plaintext = await window.crypto.subtle.decrypt(
      { name: 'AES-GCM', iv },
      key,
      ciphertext
    );
  }

```

```

        setProgress(90);

        // Step 5: Create result blob
        const blob = new Blob([plaintext]);
        setProgress(100);
        setResultBlob(blob);
        setStatus('success');

        // Auto-wipe
        wipeBuffer(fileBuffer);

        return blob;
    } catch (err) {
        console.error('Decryption error:', err);
        const isIntegrityError = err.name === 'OperationError';
        setError(
            isIntegrityError
                ? 'Dekripsi gagal! Password salah atau file telah dimodifikasi (integrity check gagal).'
                : 'Gagal mendekripsi file. Pastikan file ini terenkripsi dan password benar.'
        );
        setStatus('error');
        return null;
    }
}, [reset]);

return { encrypt, decrypt, status, progress, error, resultBlob, reset, abort };
}

/**
 * Attempt to wipe an ArrayBuffer from memory.
 */
function wipeBuffer(buffer) {
    try {
        const view = new Uint8Array(buffer);
        window.crypto.getRandomValues(view); // overwrite with random data
    } catch {
        // Buffer may already be detached (transferred), which is fine
    }
}

```

```
}
```

A.2 Kode Program src/utils/helpers.js

Deskripsi: Berisi fungsi helper utilitas untuk penanganan file dan manipulasi data. Berfungsi untuk menaksir kekuatan password secara dinamis, mengunduh file menggunakan File System Access API (`window.showSaveFilePicker`) untuk menyimpan file dengan nama aslinya secara interaktif, melakukan pemformatan ukuran byte menjadi format yang mudah dibaca, serta menghasilkan nama file output yang sesuai (menambahkan atau menghapus ekstensi `.enc`).

```
/**
 * helpers.js - Utility functions for file handling and data conversion.
 */

export function formatFileSize(bytes) {
  if (bytes === 0) return '0 Bytes';
  const k = 1024;
  const sizes = ['Bytes', 'KB', 'MB', 'GB', 'TB'];
  const i = Math.floor(Math.log(bytes) / Math.log(k));
  return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + ' ' + sizes[i];
}

export function getFileExtension(filename) {
  const parts = filename.split('.');
  if (parts.length < 2) return '';
  return parts.pop().toLowerCase();
}

export function getFileType(filename) {
  const ext = getFileExtension(filename);
  const map = {
    pdf: 'PDF Document', doc: 'Word Document', docx: 'Word Document',
    xls: 'Excel Spreadsheet', xlsx: 'Excel Spreadsheet',
    txt: 'Text File', csv: 'CSV File', json: 'JSON File',
    png: 'PNG Image', jpg: 'JPEG Image', jpeg: 'JPEG Image',
    gif: 'GIF Image', svg: 'SVG Image', webp: 'WebP Image',
    mp3: 'MP3 Audio', mp4: 'MP4 Video', zip: 'ZIP Archive',
    rar: 'RAR Archive', exe: 'Executable', enc: 'Encrypted File',
  };
}
```

```

    };
    return map[ext] || (ext ? `${ext.toUpperCase()} File` : 'Unknown
File');
  }

export function getFileIcon(filename) {
  const ext = getFileExtension(filename);
  const map = {
    pdf: '📄', doc: '📄', docx: '📄', xls: '📊', xlsx: '📊',
    txt: '📄', csv: '📊', json: '⚙️', png: '🖼️', jpeg: '🖼️',
    jpg: '🖼️', gif: '🖼️', svg: '🖼️', mp3: '🎵', mp4: '🎬',
    zip: '📦', rar: '📦', exe: '⚙️', enc: '🔒',
  };
  return map[ext] || '📄';
}

export function evaluatePasswordStrength(password) {
  if (!password) return { score: 0, label: '', color: '' };
  let score = 0;
  if (password.length >= 6) score += 1;
  if (password.length >= 10) score += 1;
  if (password.length >= 14) score += 1;
  if (/[a-z]/.test(password)) score += 0.5;
  if (/[A-Z]/.test(password)) score += 0.5;
  if (/[0-9]/.test(password)) score += 0.5;
  if (/^[a-zA-Z0-9]/.test(password)) score += 1;
  if (/^(.)\1+$/.test(password)) score = Math.max(score - 2, 0);
  const level = Math.min(Math.floor(score / 1.5), 3);
  const levels = [
    { score: 0, label: 'Lemah', color: 'weak' },
    { score: 1, label: 'Cukup', color: 'fair' },
    { score: 2, label: 'Bagus', color: 'good' },
    { score: 3, label: 'Kuat', color: 'strong' },
  ];
  return levels[level];
}

export async function downloadBlob(blob, filename) {

```



```

    // Primary method: File System Access API (works perfectly in
Edge/Chrome)
    // Shows a native "Save As" dialog with the correct filename
    if (window.showSaveFilePicker) {
        try {
            const handle = await window.showSaveFilePicker({
                suggestedName: filename,
            });
            const writable = await handle.createWritable();
            await writable.write(blob);
            await writable.close();
            return;
        } catch (err) {
            // User cancelled the save dialog
            if (err.name === 'AbortError') return;
            // If API fails for other reasons, fall through to anchor method
        }
    }

    // Fallback: anchor element method (for Firefox / unsupported
browsers)
    const downloadFile = new Blob([blob], { type: 'application/octet-
stream' });
    const url = URL.createObjectURL(downloadFile);
    const a = document.createElement('a');
    a.style.display = 'none';
    a.href = url;
    a.download = filename;
    document.body.appendChild(a);
    a.click();
    setTimeout(() => {
        document.body.removeChild(a);
        URL.revokeObjectURL(url);
    }, 1000);
}

export function generateOutputFilename(originalName, mode) {
    if (mode === 'encrypt') return `${originalName}.enc`;
    if (originalName.endsWith('.enc')) return originalName.slice(0, -4);
    return `decrypted_${originalName}`;
}

```

```
export function delay(ms) {  
  return new Promise(resolve => setTimeout(resolve, ms));  
}
```

A.3 Kode Program src/App.jsx

Deskripsi: Komponen utama (entri utama UI/UX) dari aplikasi React. Berfungsi sebagai pengelola status (state manager) keseluruhan aplikasi, seperti mode operasi aktif (enkripsi/dekripsi), melacak file yang dipilih, mendengarkan input password, mengoordinasikan interaksi fungsi kriptografi (`useCrypto`), menampilkan progress bar, pesan kesalahan, instruksi penanganan file pasca-enkripsi, dan tombol aksi eksekusi.

```
import { useState, useCallback } from 'react';  
import Dropzone from '../components/Dropzone';  
import PasswordField from '../components/PasswordField';  
import FileInfo from '../components/FileInfo';  
import { useCrypto } from '../hooks/useCrypto';  
import { downloadBlob, generateOutputFilename } from '../utils/helpers';  
  
export default function App() {  
  const [mode, setMode] = useState('encrypt');  
  const [file, setFile] = useState(null);  
  const [password, setPassword] = useState('');  
  const [outputFilename, setOutputFilename] = useState('');  
  const { encrypt, decrypt, status, progress, error, resultBlob, reset  
} = useCrypto();  
  
  const isProcessing = status === 'processing';  
  const isSuccess = status === 'success';  
  const isError = status === 'error';  
  const canProcess = file && password.length >= 1 && !isProcessing;  
  
  const handleFileSelect = useCallback((f) => {  
    setFile(f);  
    reset();  
  }, [reset]);  
  
  const handleRemoveFile = useCallback(() => {  
    setFile(null);  
    setPassword('');
```

```

        reset();
    }, [reset]);

    const handleModeChange = useCallback((newMode) => {
        setMode(newMode);
        setFile(null);
        setPassword('');
        reset();
    }, [reset]);

    const handleProcess = useCallback(async () => {
        if (!canProcess) return;
        const blob = mode === 'encrypt'
            ? await encrypt(file, password)
            : await decrypt(file, password);
        if (blob) {
            const outName = generateOutputFilename(file.name, mode);
            setOutputFilename(outName);
            await downloadBlob(blob, outName);
        }
    }, [canProcess, mode, file, password, encrypt, decrypt]);

    const handleReset = useCallback(() => {
        setFile(null);
        setPassword('');
        reset();
    }, [reset]);

    return (
        <div className="min-h-screen relative">
            {/* Animated background */}
            <div className="bg-grid" />
            <div className="bg-glow bg-glow-1" />
            <div className="bg-glow bg-glow-2" />
            <div className="bg-glow bg-glow-3" />

            {/* Main content */}
            <div className="relative z-10 flex flex-col items-center min-h-
screen px-4 py-8 md:py-12">
                {/* Header */}

```

```

<header className="text-center mb-8 md:mb-12 animate-fade-in-up" id="app-header">
  <div className="flex items-center justify-center gap-3 mb-4">
    <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-primary-500 to-accent-500 flex items-center justify-center shadow-lg shadow-primary-500/20">
      <svg className="w-6 h-6 text-white" fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth={2}>
        <path strokeLinecap="round" strokeLinejoin="round" d="M16.5 10.5V6.75a4.5 4.5 0 10-9 0v3.75m-.75 11.25h10.5a2.25 2.25 0 02.25-2.25v-6.75a2.25 2.25 0 00-2.25-2.25H6.75a2.25 2.25 0 00-2.25 2.25v6.75a2.25 2.25 0 02.25 2.25z" />
      </svg>
    </div>
  </div>
  <h1 className="text-3xl md:text-4xl font-bold bg-gradient-to-r from-primary-300 via-primary-200 to-accent-400 bg-clip-text text-transparent">
    SECURE-WEB-LOCK
  </h1>
  <p className="text-surface-200/50 text-sm md:text-base mt-2 max-w-md mx-auto">
    Enkripsi file dengan AES-GCM 256-bit langsung di browser Anda.
  <br />
  <span className="text-accent-400/70 font-medium">Zero Knowledge • 100% Client-Side</span>
</p>
</header>

{/* Main Card */}
<main className="w-full max-w-xl animate-fade-in-up" style={{ animationDelay: '0.15s' }}>
  <div className="glass-card p-6 md:p-8 space-y-6" id="main-card">

    {/* Mode Toggle */}
    <div className="mode-toggle" id="mode-toggle">
      <button
        className={`mode-btn ${mode === 'encrypt' ? 'active' : ''}`}

```

```

        onClick={() => handleModeChange('encrypt')}
        id="btn-mode-encrypt"
      >
        <svg className="w-4 h-4" fill="none" viewBox="0 0 24
24" stroke="currentColor" strokeWidth={2}>
          <path strokeLinecap="round" strokeLinejoin="round"
d="M16.5 10.5V6.75a4.5 4.5 0 10-9 0v3.75m-.75 11.25h10.5a2.25 2.25 0
002.25-2.25v-6.75a2.25 2.25 0 00-2.25-2.25H6.75a2.25 2.25 0 00-2.25
2.25v6.75a2.25 2.25 0 002.25 2.25z" />
        </svg>
        Enkripsi
      </button>
      <button
        className={`mode-btn ${mode === 'decrypt' ? 'active' :
''}`}
        onClick={() => handleModeChange('decrypt')}
        id="btn-mode-decrypt"
      >
        <svg className="w-4 h-4" fill="none" viewBox="0 0 24
24" stroke="currentColor" strokeWidth={2}>
          <path strokeLinecap="round" strokeLinejoin="round"
d="M13.5 10.5V6.75a4.5 4.5 0 119 0v3.75M3.75 21.75h10.5a2.25 2.25 0 002.25-
2.25v-6.75a2.25 2.25 0 00-2.25-2.25H3.75a2.25 2.25 0 00-2.25 2.25v6.75a2.25
2.25 0 002.25 2.25z" />
        </svg>
        Dekripsi
      </button>
    </div>

    {/* Dropzone or File Info */}
    {!file ? (
      <Dropzone
        onFileSelect={handleFileSelect}
        disabled={isProcessing} />
    ) : (
      <FileInfo
        file={file}
        onRemove={handleRemoveFile}
        disabled={isProcessing} />
    )}

    {/* Password Field */}
    {file && (
      <PasswordField

```

```

        password={password}
        onChange={setPassword}
        disabled={isProcessing}
        mode={mode}
      />
    )}

    {/* Progress Bar */}
    {isProcessing && (
      <div className="space-y-2 animate-fade-in" id="progress-
section">
        <div className="flex items-center justify-between text-
xs">
          <span className="text-surface-200/50">
            {mode === 'encrypt' ? 'Mengenkripsi...' :
'Mendekripsi...'}
          </span>
          <span className="text-primary-400 font-mono font-
medium">{Math.round(progress)}%</span>
        </div>
        <div className="progress-track">
          <div className="progress-fill" style={{ width:
`${progress}%` }} />
        </div>
      </div>
    )}

    {/* Error Message */}
    {isError && error && (
      <div className="animate-shake" id="error-message">
        <div className="flex items-start gap-3 p-4 rounded-xl
bg-danger-500/8 border border-danger-500/15">
          <svg className="w-5 h-5 text-danger-400 flex-shrink-
0 mt-0.5" fill="none" viewBox="0 0 24 24" stroke="currentColor"
strokeWidth={2}>
            <path strokeLinecap="round" strokeLinejoin="round"
d="M12 9v3.75m9-.75a9 9 0 11-18 0 9 9 0 0118 0zm-9 3.75h.008v.008H12v-
.008z" />
          </svg>
          <div>

```

```

                <p className="text-danger-400 text-sm font-
medium">Gagal!</p>
                <p className="text-danger-400/70 text-xs mt-
0.5">{error}</p>
            </div>
        </div>
    </div>
    )}

    {/* Success Message */}
    {isSuccess && (
        <div className="animate-fade-in space-y-3" id="success-
message">
            <div className="flex items-start gap-3 p-4 rounded-xl
bg-accent-500/8 border border-accent-500/15">
                <svg className="w-5 h-5 text-accent-400 flex-shrink-
0 mt-0.5" fill="none" viewBox="0 0 24 24" stroke="currentColor"
strokeWidth={2}>
                    <path strokeLinecap="round" strokeLinejoin="round"
d="M9 12.75L11.25 15 15 9.75M21 12a9 9 0 11-18 0 9 9 0 0118 0z" />
                </svg>
                <div className="flex-1 min-w-0">
                    <p className="text-accent-400 text-sm font-
medium">Berhasil!</p>
                    <p className="text-accent-400/70 text-xs mt-1">
                        {mode === 'encrypt'
                            ? 'File berhasil dienkripsi dan sedang
didownload.'
                            : 'File berhasil didekripsi dan sedang
didownload.'}
                    </p>
                    {outputFilename && (
                        <p className="text-surface-200/50 text-xs mt-2
font-mono bg-surface-800/50 rounded-lg px-3 py-1.5 truncate"
title={outputFilename}>
                            📁 {outputFilename}
                        </p>
                    )}
                </div>
            </div>
        </div>
    )}

```

```

        { /* Instructions after encryption */
        {mode === 'encrypt' && (
            <div className="p-4 rounded-xl bg-primary-500/5
border border-primary-500/10">
                <p className="text-primary-300 text-xs font-medium
mb-2"><img alt="document icon" data-bbox="191 208 208 225"/> Cara mendekripsi file:</p>
                <ol className="text-surface-200/50 text-xs space-
y-1 list-decimal list-inside">
                    <li>Pindah ke mode <strong className="text-
primary-300">Dekripsi</strong></li>
                    <li>Upload file <strong className="text-primary-
300 font-mono">.enc</strong> yang baru didownload</li>
                    <li>Masukkan <strong className="text-primary-
300">password yang sama</strong></li>
                    <li>Klik tombol <strong className="text-primary-
300">Dekripsi File</strong></li>
                </ol>
            </div>
        )}
    </div>
)}

{ /* Action Buttons */
{file && (
    <div className="flex flex-col gap-3 animate-fade-in"
id="action-buttons">
        {!isSuccess ? (
            <div className="flex gap-3">
                <button
                    className="btn-primary flex-1"
                    onClick={handleProcess}
                    disabled={!canProcess}
                    id="btn-process"
                >
                    {isProcessing ? (
                        <>
                            <svg className="w-4 h-4 animate-spin-slow"
fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth={2}>

```



```

                                <path                                strokeLinecap="round"
strokeLinejoin="round" d="M16.023 9.348h4.992v-.001M2.985 19.644v-4.992m0
0h4.992m-4.993 0l3.181 3.183a8.25 8.25 0 0013.803-3.7M4.031 9.865a8.25 8.25
0 0113.803-3.7l3.181 3.182m0-4.991v4.99" />
                                </svg>
                                Memproses...
                                </>
                                ) : (
                                <>
                                {mode === 'encrypt' ? (
                                <svg className="w-4 h-4" fill="none"
viewBox="0 0 24 24" stroke="currentColor" strokeWidth={2}>
                                <path                                strokeLinecap="round"
strokeLinejoin="round" d="M16.5 10.5V6.75a4.5 4.5 0 10-9 0v3.75m-.75
11.25h10.5a2.25 2.25 0 002.25-2.25v-6.75a2.25 2.25 0 00-2.25-
2.25H6.75a2.25 2.25 0 00-2.25 2.25v6.75a2.25 2.25 0 002.25 2.25z" />
                                </svg>
                                ) : (
                                <svg className="w-4 h-4" fill="none"
viewBox="0 0 24 24" stroke="currentColor" strokeWidth={2}>
                                <path                                strokeLinecap="round"
strokeLinejoin="round" d="M13.5 10.5V6.75a4.5 4.5 0 119 0v3.75M3.75
21.75h10.5a2.25 2.25 0 002.25-2.25v-6.75a2.25 2.25 0 00-2.25-
2.25H3.75a2.25 2.25 0 00-2.25 2.25v6.75a2.25 2.25 0 002.25 2.25z" />
                                </svg>
                                )}
                                {mode === 'encrypt' ? 'Enkripsi File' :
'Dekripsi File'}
                                </>
                                )}
                                </button>
                                {!isProcessing && (
                                <button
                                className="btn-danger"
                                onClick={handleReset}
                                id="btn-reset"
                                >
                                <svg className="w-4 h-4" fill="none" viewBox="0
0 24 24" stroke="currentColor" strokeWidth={2}>
                                <path                                strokeLinecap="round"
strokeLinejoin="round" d="M6 18L18 6M6 6l12 12" />

```

```

        </svg>
      </button>
    )}
  </div>
) : (
  <div className="flex gap-3">
    {mode === 'encrypt' && (
      <button
        className="btn-primary flex-1"
        onClick={() => handleModeChange('decrypt')}
        id="btn-go-decrypt"
      >
        <svg className="w-4 h-4" fill="none" viewBox="0
0 24 24" stroke="currentColor" strokeWidth={2}>
          <path
            strokeLinecap="round"
            strokeLinejoin="round" d="M13.5 10.5V6.75a4.5 4.5 0 119 0v3.75M3.75
21.75h10.5a2.25 2.25 0 002.25-2.25v-6.75a2.25 2.25 0 00-2.25-
2.25H3.75a2.25 2.25 0 00-2.25 2.25v6.75a2.25 2.25 0 002.25 2.25z" />
          </svg>
          Dekripsi File .enc
        </button>
      )}
      <button
        className="btn-secondary flex-1"
        onClick={handleReset}
        id="btn-new-file"
      >
        <svg className="w-4 h-4" fill="none" viewBox="0 0
24 24" stroke="currentColor" strokeWidth={2}>
          <path
            strokeLinecap="round" d="M12 4.5v15m7.5-7.5h-15" />
          </svg>
          Proses File Lain
        </button>
      </div>
    )}
  </div>
)}
</div>

{/* Security Banner */}

```

```

        <div className="security-banner mt-4 animate-fade-in"
style={{ animationDelay: '0.3s' }} id="security-banner">
            <svg className="w-4 h-4 flex-shrink-0" fill="none"
viewBox="0 0 24 24" stroke="currentColor" strokeWidth={2}>
                <path strokeLinecap="round" strokeLinejoin="round" d="M9
12.75L11.25 15 15 9.75m-3-7.036A11.959 11.959 0 013.598 6 11.99 11.99 0
003 9.749c0 5.592 3.824 10.29 9 11.623 5.176-1.332 9-6.03 9-11.622 0-1.31-
.21-2.571-.598-3.751h-.152c-3.196 0-6.1-1.248-8.25-3.285z" />
            </svg>
            <span className="text-xs">
                Data Anda tidak pernah meninggalkan browser – <strong>100%
client-side encryption</strong>
            </span>
        </div>

        { /* Tech Info */ }
        <div className="mt-6 flex flex-wrap items-center justify-
center gap-2 animate-fade-in" style={{ animationDelay: '0.4s' }} id="tech-
badges">
            <span className="badge badge-info"><img alt="lock icon" data-bbox="675 475 695 490" /> AES-GCM 256-
bit</span>
            <span className="badge badge-info"><img alt="key icon" data-bbox="675 515 695 530" /> PBKDF2 100K
iterations</span>
            <span className="badge badge-info"><img alt="shield icon" data-bbox="630 560 650 575" /> SHA-256</span>
        </div>
    </main>

    { /* Footer */ }
    <footer className="mt-auto pt-8 text-center text-surface-200/25
text-xs animate-fade-in" style={{ animationDelay: '0.5s' }} id="app-
footer">
        <p>SECURE-WEB-LOCK &copy; 2026 – Powered by Web Crypto API</p>
        <p className="mt-1">Built with React + Vite • Zero Knowledge
Architecture</p>
    </footer>
</div>
</div>

);
}

```

A.4 Kode Program src/components/Dropzone.jsx

Deskripsi: Komponen area seret dan lepas (drag & drop) file. Berfungsi menangani input berkas dari pengguna, mendeteksi efek hovering visual ketika berkas ditarik di atas area drop, serta memicu pemuatan berkas ke aplikasi baik dengan cara diseret maupun diklik (memanfaatkan input file tersembunyi).

```
import { useCallback, useState, useRef } from 'react';

export default function Dropzone({ onFileSelect, disabled }) {
  const [isDragging, setIsDragging] = useState(false);
  const inputRef = useRef(null);

  const handleDragOver = useCallback((e) => {
    e.preventDefault();
    e.stopPropagation();
    if (!disabled) setIsDragging(true);
  }, [disabled]);

  const handleDragLeave = useCallback((e) => {
    e.preventDefault();
    e.stopPropagation();
    setIsDragging(false);
  }, []);

  const handleDrop = useCallback((e) => {
    e.preventDefault();
    e.stopPropagation();
    setIsDragging(false);
    if (disabled) return;
    const files = e.dataTransfer.files;
    if (files.length > 0) onFileSelect(files[0]);
  }, [disabled, onFileSelect]);

  const handleClick = () => {
    if (!disabled && inputRef.current) inputRef.current.click();
  };

  const handleChange = (e) => {
    if (e.target.files.length > 0) {
```

```

        onFileSelect(e.target.files[0]);
        e.target.value = '';
    }
};

return (
    <div
        id="dropzone"
        className={`dropzone p-8 md:p-12 text-center ${isDragging ?
'active' : ''} ${disabled ? 'opacity-40 cursor-not-allowed' : ''}`}
        onDragOver={handleDragOver}
        onDragLeave={handleDragLeave}
        onDrop={handleDrop}
        onClick={handleClick}
        role="button"
        tabIndex={0}
        aria-label="Area drag and drop file"
    >
        <input
            ref={inputRef}
            type="file"
            className="hidden"
            onChange={handleChange}
            disabled={disabled}
            id="file-input"
        />

        <div className="relative z-10 flex flex-col items-center gap-4">
            {/* Upload icon */}
            <div className={`w-16 h-16 rounded-2xl flex items-center
justify-center transition-all duration-300 ${isDragging ? 'bg-primary-
500/20 scale-110' : 'bg-primary-500/10'}`}>
                <svg className={`w-8 h-8 transition-colors duration-300
${isDragging ? 'text-primary-300' : 'text-primary-400'}`} fill="none"
viewBox="0 0 24 24" stroke="currentColor" strokeWidth={1.5}>
                    <path strokeLinecap="round" strokeLinejoin="round" d="M3
16.5v2.25A2.25 2.25 0 005.25 21h13.5A2.25 2.25 0 0021 18.75V16.5m-13.5-
9L12 3m0 0 14.5 4.5M12 3v13.5" />
                </svg>
            </div>
        </div>
    </div>

```

```

        <div>
          <p className="text-surface-100 font-medium text-lg mb-1">
            {isDragging ? 'Lepaskan file di sini...' : 'Drag & drop file
di sini'}
          </p>
          <p className="text-surface-200/50 text-sm">
            atau <span className="text-primary-400 underline underline-
offset-2">klik untuk memilih file</span>
          </p>
        </div>

        <p className="text-surface-200/30 text-xs">
          Mendukung semua jenis file • Tanpa batas ukuran
        </p>
      </div>
    </div>
  );
}

```

A.5 Kode Program src/components/PasswordField.jsx

Deskripsi: Komponen kolom input kata sandi yang disesuaikan (custom password field). Berfungsi untuk mengambil masukan kata sandi enkripsi/dekripsi, menyediakan fitur untuk melihat/menyembunyikan kata sandi secara interaktif, serta menampilkan indikator visual tingkat kekuatan password (lemah, cukup, bagus, kuat) secara real-time pada mode enkripsi.

```

import { useState, useMemo } from 'react';
import { evaluatePasswordStrength } from '../utils/helpers';

export default function PasswordField({ password, onChange, disabled,
mode }) {
  const [showPassword, setShowPassword] = useState(false);
  const strength = useMemo(() => evaluatePasswordStrength(password),
[password]);

  return (
    <div className="space-y-3 animate-fade-in" id="password-section">
      <label className="block text-sm font-medium text-surface-200/70"
htmlFor="password-input">

```

```

        {mode === 'encrypt' ? '🔑 Password Enkripsi' : '🔑 Password
Dekripsi'}}
    </label>

    <div className="relative">
        <input
            id="password-input"
            type={showPassword ? 'text' : 'password'}
            value={password}
            onChange={(e) => onChange(e.target.value)}
            placeholder={mode === 'encrypt' ? 'Masukkan password untuk
enkripsi...' : 'Masukkan password yang sama saat enkripsi...'}
            className="input-field pr-12 font-mono"
            disabled={disabled}
            autoComplete="off"
        />
        <button
            type="button"
            onClick={() => setShowPassword(!showPassword)}
            className="absolute right-3 top-1/2 -translate-y-1/2 text-
surface-200/40 hover:text-surface-200/80 transition-colors p-1"
            tabIndex={-1}
            id="toggle-password-visibility"
            aria-label={showPassword ? 'Sembunyikan password' :
'Tampilkan password'}
        >
            {showPassword ? (
                <svg className="w-5 h-5" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth={1.5}>
                    <path strokeLinecap="round" strokeLinejoin="round"
d="M3.98 8.223A10.477 10.477 0 01.934 12C3.226 16.338 7.244 19.5 12
19.5c.993 0 1.953-.138 2.863-.395M6.228 6.228A10.45 10.45 0 0112 4.5c4.756
0 8.773 3.162 10.065 7.498a10.523 10.523 0 01-4.293 5.774M6.228 6.228L3
3m3.228 3.228l3.65 3.65m7.894 7.894L21 21m-3.228-3.228l-3.65-3.65m0 0a3 3
0 10-4.243-4.243m4.242 4.242L9.88 9.88" />
                </svg>
            ) : (
                <svg className="w-5 h-5" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth={1.5}>

```

```

        <path      strokeLinecap="round"      strokeLinejoin="round"
d="M2.036 12.322a1.012 1.012 0 010-.639C3.423 7.51 7.36 4.5 12 4.5c4.638 0
8.573 3.007 9.963 7.178.07.207.07.431 0 .639C20.577 16.49 16.64 19.5 12
19.5c-4.638 0-8.573-3.007-9.963-7.178z" />
        <path strokeLinecap="round" strokeLinejoin="round" d="M15
12a3 3 0 11-6 0 3 3 0 016 0z" />
    </svg>
  )}
</button>
</div>

{/* Password strength meter – only show on encrypt mode */}
{mode === 'encrypt' && password && (
  <div className="space-y-1.5 animate-fade-in">
    <div className="strength-meter">
      {[0, 1, 2, 3].map((i) => (
        <div
          key={i}
          className={`strength-bar  ${i  <=  strength.score  ?
`active-${strength.color}` : ''}`}
        />
      ))}
    </div>
    <p className={`text-xs font-medium ${
      strength.color === 'weak' ? 'text-danger-400' :
      strength.color === 'fair' ? 'text-warning-400' :
      strength.color === 'good' ? 'text-primary-400' :
      'text-accent-400'
    }`}>
      Kekuatan: {strength.label}
    </p>
  </div>
)}
</div>
);
}

```

A.6 Kode Program src/components/FileInfo.jsx

Deskripsi: Komponen yang menampilkan informasi mendetail tentang file yang telah berhasil dimuat oleh dropzone. Komponen ini menampilkan nama file, tipe file berdasarkan

ekstensinya, ukuran file yang telah terformat ke satuan yang sesuai (misal: KB, MB), ikon representatif berkas, serta menyediakan tombol hapus (trash/remove) untuk mengembalikan pilihan berkas ke kondisi semula.

```
import { formatFileSize, getFileType, getFileIcon } from
'../utils/helpers';

export default function FileInfo({ file, onRemove, disabled }) {
  if (!file) return null;

  const icon = getFileIcon(file.name);
  const type = getFileType(file.name);
  const size = formatFileSize(file.size);

  return (
    <div className="animate-slide-in-right" id="file-info">
      <div className="flex items-center gap-4 p-4 rounded-xl bg-
primary-500/5 border border-primary-500/10">
        {/* File icon */}
        <div className="w-12 h-12 rounded-xl bg-primary-500/10 flex
items-center justify-center text-2xl flex-shrink-0">
          {icon}
        </div>

        {/* File details */}
        <div className="flex-1 min-w-0">
          <p className="text-surface-100 font-medium truncate text-sm"
title={file.name}>
            {file.name}
          </p>
          <div className="flex items-center gap-2 mt-0.5">
            <span
              className="text-surface-200/40
text-
xs">{type}</span>
            <span className="text-surface-200/20">•</span>
            <span
              className="text-surface-200/40
text-xs
font-
mono">{size}</span>
          </div>
        </div>

        {/* Remove button */}
```

```

        {!disabled && (
            <button
                onClick={onRemove}
                className="w-8 h-8 rounded-lg flex items-center justify-
center text-surface-200/30 hover:text-danger-400 hover:bg-danger-500/10
transition-all"
                id="remove-file-btn"
                aria-label="Hapus file"
            >
                <svg className="w-4 h-4" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth={2}>
                    <path strokeLinecap="round" strokeLinejoin="round" d="M6
18L18 6M6 6l12 12" />
                </svg>
            </button>
        )}
    </div>
</div>
);
}

```